Universidade Federal da Fronteira Sul
Curso de Ciência da Computação

Redes de Computadores

Prof. Marco Aurélio Spohn

marco.spohn@uffs.edu.br

Esta apresentação contém slides fornecidos pela Editora Pearson como material de apoio ao Professor do livro "Redes de Computadores e a Internet: uma abordagem top-down", bem como material da disciplina "Computer Networks" ministrada pelo Prof. J. J. Garcia-Luna-Aceves da University of California at Santa Cruz

Camada de aplicação

- 2.1 Princípios de aplicações em rede de computadores
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

Algumas aplicações de rede

- Correio Eletrônico (e-mail)
- Hipertexto (http://www...)
- Mensagem instantânea
- Login remoto (e.g., ssh)
- Compartilhamento de Arquivos Entre-Pares (P2P *file sharing*)
- Jogos multi-usuário em Rede
- *Streaming stored videoclipes*
- Telefonia via Internet (VoIP)
- Videoconferência em tempo real
- Grades Computacionais (*grid computing*)

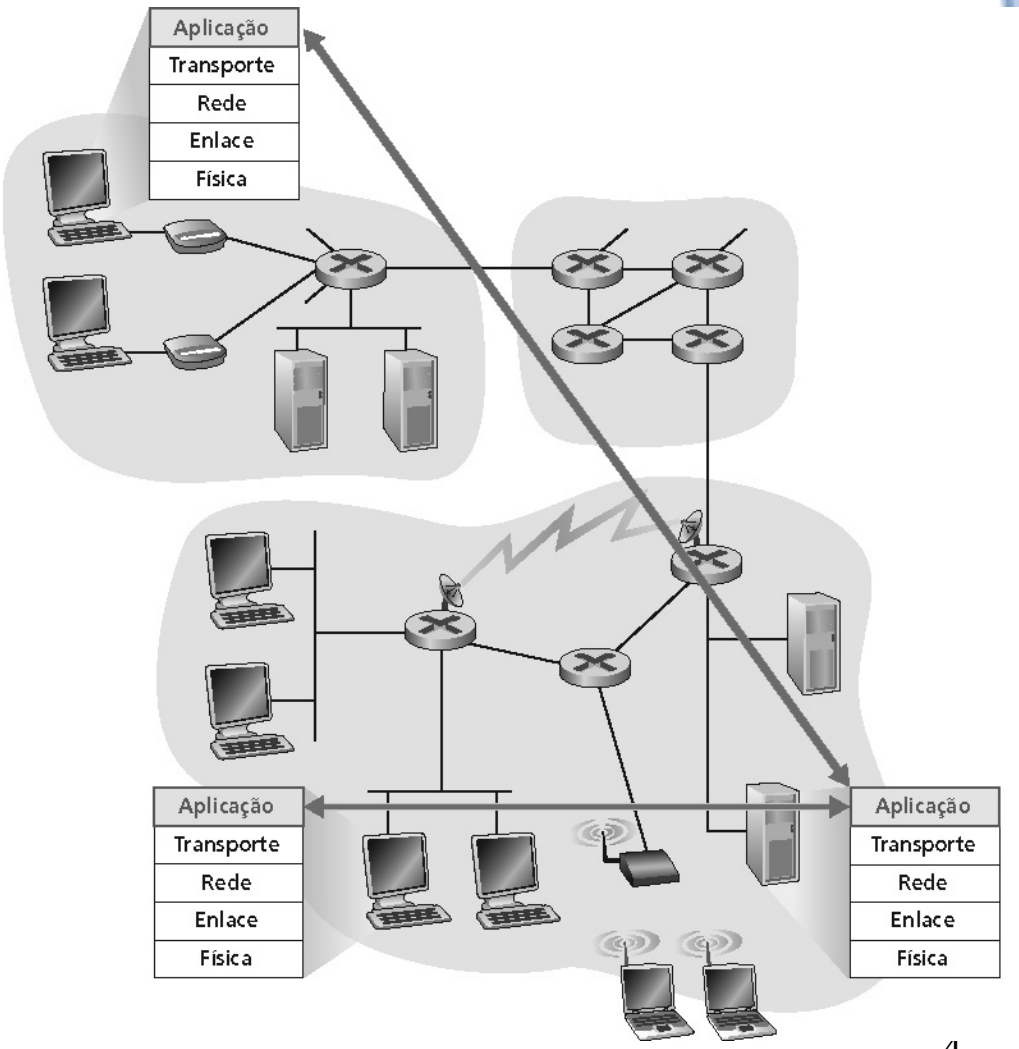
Criando uma nova aplicação de rede

Escrever programas que

- Executem sobre diferentes sistemas finais e
- Se comuniquem através de uma rede.
- Ex.: *Software* de servidor Web se comunicando com *software* do navegador (*browser*).

Nenhum software é escrito para dispositivos no núcleo da rede

- Dispositivos do núcleo da rede não trabalham na camada de aplicação
- Esta estrutura permite um rápido desenvolvimento de aplicação



Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 HTTP
- 2.3 FTP
- 2.4 Correio eletrônico: SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos Entre-Pares (peer-to-peer, P2P)
- 2.7 Programação de *socket* com TCP
- 2.8 Programação de *socket* com UDP

Arquiteturas de aplicação

- Cliente-servidor
- Entre-Pares (**Peer-to-peer,** P2P)
- Híbrida de cliente-servidor e P2P

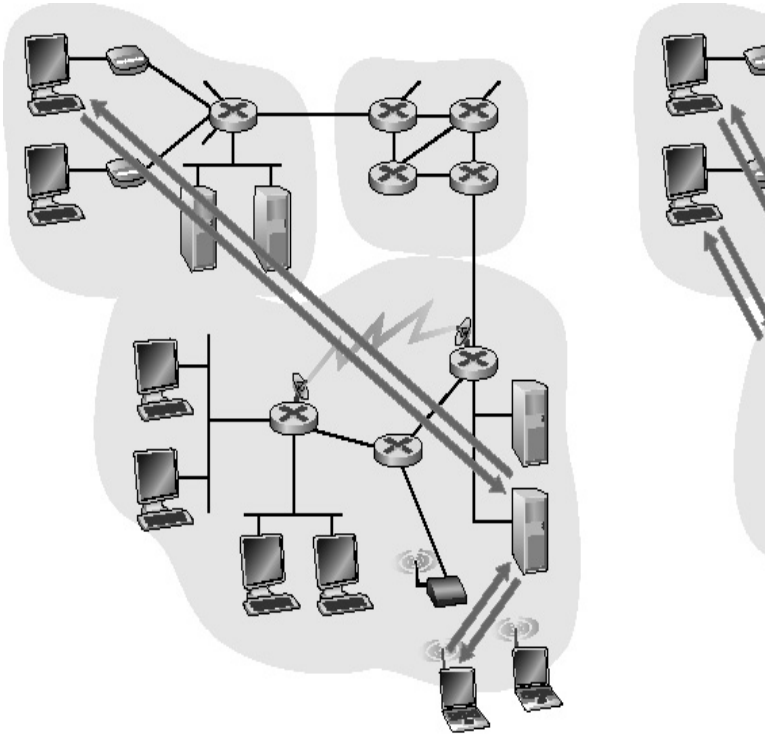
Arquitetura cliente-servidor

Servidor:

- Hospedeiro sempre ativo
- Endereço IP permanente
- Fornece serviços solicitados pelo cliente

Clientes:

- Comunicam-se com o servidor
- Pode ser conectado intermitentemente
- Pode ter endereço IP dinâmico
- Não se comunicam diretamente uns com os outros



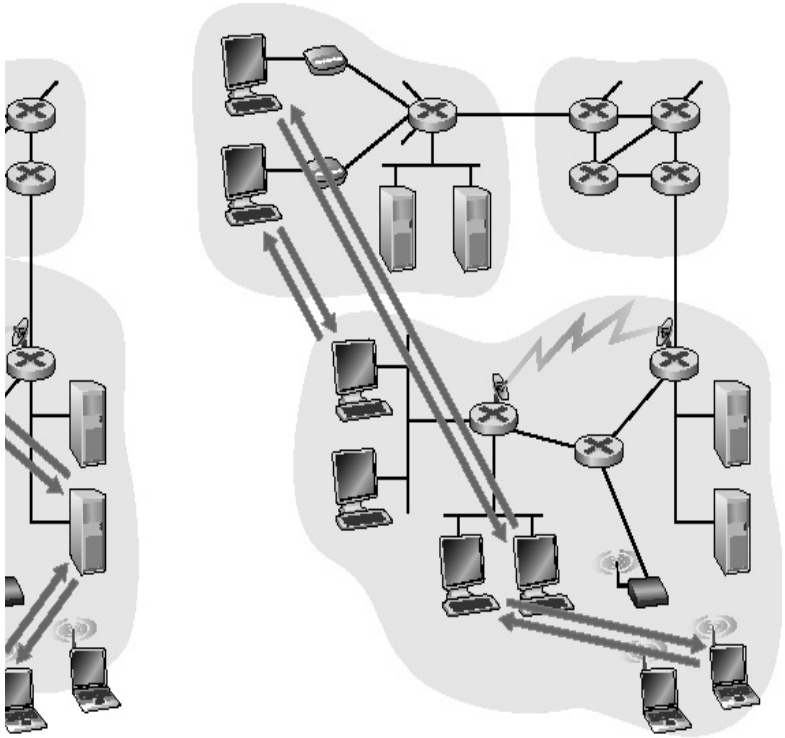
a. Aplicação cliente-servidor b. Aplicação

Arquitetura P2P pura

- Nem sempre no servidor
- Sistemas finais arbitrários comunicam-se diretamente
- Pares são intermitentemente conectados e trocam endereços IP
- Ex.: Gnutella (lê-se *newtella)*

Altamente escaláveis mas difíceis

de gerenciar



b. Aplicação P2P

Híbrida de cliente-servidor e P2P

Napster

- Transferência de arquivo P2P
- Busca centralizada de arquivos:
- Conteúdo de registro dos pares no servidor central
- Consulta de pares no mesmo servidor central para localizar o conteúdo

Instant messaging

- Bate-papo entre dois usuários é P2P
- Detecção/localização de presença é centralizada:
- Usuário registra seu endereço IP com o servidor central quando fica *on-line*
- Usuário contata o servidor central para encontrar endereços IP dos “amigos”

Comunicação de processos

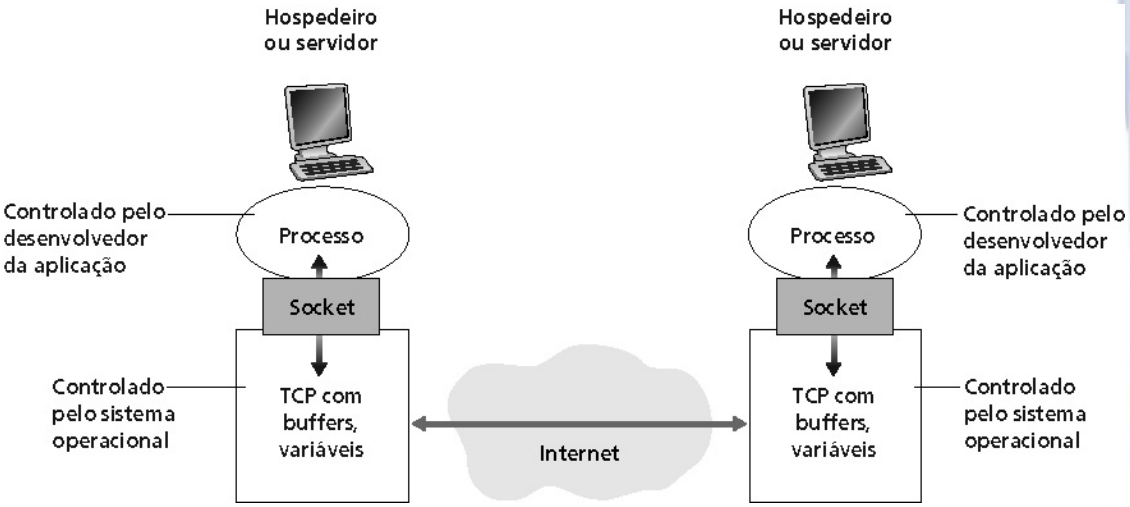
Processo: programa executando num hospedeiro

- Dentro do mesmo hospedeiro: dois processos se comunicam usando comunicação interprocesso (definido pelo Sistema Operacional)
- Processos em diferentes hospedeiros se comunicam por meio de troca de mensagens
- Processo cliente: processo que inicia a comunicação
- Processo servidor: processo que espera para ser contatado

Nota: aplicações com arquiteturas P2P possuem processos cliente e processos servidor

Sockets

- Um processo envia/recebe mensagens para/de seu *socket*
- O *socket* é análogo a uma porta
  - O processo de envio empurra a mensagem para fora da porta
  - O processo de envio confia na infra-estrutura de transporte no outro lado da porta que leva a mensagem para o *socket* no processo de recepção.



Processos de endereçamento

- Para um processo receber mensagens, ele deve ter um identificador
- Um hospedeiro possui um único endereço IP de 32 bits (assumindo IPv4)
- P.: O endereço IP do hospedeiro onde o processo está executando é suficiente para identificar o processo?
- R.: Não, muitos processos podem estar em execução no mesmo hospedeiro.
- O identificador inclui o endereço IP e o número da porta associada ao processo no hospedeiro
- Exemplos de números de porta:
  - Servidor HTTP: 80
  - Servidor de Correio: 25

O protocolo da camada de aplicação define

- Tipo das mensagens trocadas, mensagens de requisição e resposta
- Sintaxe dos tipos de mensagem: os campos nas mensagens e como são delineados
- Semântica dos campos, ou seja, significado da informação nos campos
- Regras para quando e como os processos enviam e respondem às mensagens

Protocolos de domínio público:

- Definidos nas RFCs
- Recomendados para interoperabilidade
- Ex.: HTTP, SMTP, DNS, SSH

Protocolos proprietários:

- Ex.: KaZaA

De qual serviço de transporte uma aplicação necessita?

Perda de dados

- Algumas aplicações (ex.: áudio) podem tolerar alguma perda
- Outras aplicações (ex.: transferência de arquivos, sessão remota) exigem transferência de dados 100% confiável

Temporização

- Algumas aplicações (ex.: telefonia Internet, jogos interativos) exigem baixos atrasos para serem “efetivos”

Banda passante

- Algumas aplicações (ex.: multimídia) exigem uma banda mínima para serem “efetivas”
- Outras aplicações (“aplicações elásticas”) melhoram quando a banda disponível aumenta

Requisitos de transporte de aplicações comuns

Aplicação	Perdas	Banda	Sensível ao atraso
file transfer	sem perdas	elástica	não
e-mail	sem perdas	elástica	não
Web documents	sem perdas	elástica	não
real-time áudio/vídeo	tolerante	aúdio: 5 Kb-1 Mb	sim, 100's mseg
stored áudio/video	tolerante	vídeo:10 Kb-5 Mb	sim, segundos
jogos interativos	tolerante		sim, 100's mseg
e-business	sem perda	igual à anterior kbps	sim
		elástica	

Serviços dos protocolos de transporte da Internet

Serviço TCP:

- **Orientado à conexão:** conexão requerida entre processos cliente e servidor
- **Transporte confiável** entre os processos de envio e recepção
- **Controle de fluxo:** o transmissor não sobrecarrega o receptor
- **Controle de congestionamento:** protege a rede do excesso de tráfego

  Não oferece: garantias de temporização e de banda mínima

Serviço UDP:

- Transferência de dados não confiável entre os processos transmissor e receptor
- Não oferece: estabelecimento de conexão, controle de fluxo e de congestionamento, garantia de temporização e de banda mínima.

P.: Por que ambos? Por que existe o UDP?

Aplicação e protocolos de transporte da Internet

Aplicação	Protocolo de aplicação	Protocolo de transporte
e-mail	smtp [RFC 821]	TCP
acesso de terminais remotos	telnet [RFC 854]	TCP
Web	http [RFC 2068]	TCP
transferência de arquivos	ftp [RFC 959]	TCP
streaming multimídia	RTP ou proprietário (ex.: RealNetworks)	TCP ou UDP
servidor de arquivos remoto	NFS	TCP ou UDP
telefonia Internet	RTP ou proprietário (ex.: Vocaltec)	tipicamente UDP

Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

Web e HTTP

Primeiro alguns jargões

- Página Web consiste de objetos
- Objeto pode ser arquivo HTML, imagem JPEG, Java *applet*, arquivo de áudio,...
- A página Web consiste de arquivo-HTML base que inclui vários objetos referenciados
- Cada objeto é endereçado por um *Universal Resource Locator* (URL)
- Exemplo de URL:

$$\underbrace{\texttt{www.someschool.edu}}_{\text{Nome do hospedeiro}}\underbrace{\texttt{/someDept/pic.gif}}_{\text{Nome do caminho}}$$

Visão geral do HTTP

HTTP: *hypertext transfer protocol*

- Protocolo da camada de aplicação da Web
- Modelo cliente/servidor
  - Cliente: navegador que solicita, recebe e apresenta objetos da Web
  - Servidor: envia objetos em resposta a pedidos
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068

Visão geral do HTTP

Utiliza TCP:

- Cliente inicia conexão TCP (cria *socket*) para o servidor na porta 80
- Servidor aceita uma conexão TCP do cliente
- mensagens HTTP (mensagens do protocolo de camada de aplicação) são trocadas entre o *browser* (cliente HTTP) e o servidor Web (servidor HTTP)
- A conexão TCP é fechada

HTTP é "stateless"

- Por *default*, o servidor não mantém informação sobre os pedidos passados pelos clientes

Protocolos que mantêm informações de "estado" são complexos!

- Histórico do passado (estado) deve ser mantido
- Se o servidor/cliente quebra, suas visões de "estado" podem ser inconsistentes, devendo ser reconciliadas

Conexões HTTP

HTTP não persistente

- No máximo, um objeto é enviado sobre uma conexão TCP
- O HTTP/1.0 utiliza HTTP não persistente

HTTP persistente

- Múltiplos objetos podem ser enviados sobre uma conexão
- TCP entre o cliente e o servidor
- O HTTP/1.1 utiliza conexões persistentes em seu modo padrão

HTTP não persistente

Usuário entra com a URL: (contém texto,referências a 10 imagens jpeg)

`www.someSchool.edu/someDepartment/home.index`

1a. Cliente HTTP inicia conexão TCP ao servidor HTTP (processo) em www.someSchool.edu. Porta 80 é a *default* para o servidor HTTP.

1b. Servidor HTTP no hospedeiro www.someSchool.edu esperando pela conexão TCP na porta 80. "Aceita" conexão, notificando o cliente

2. Cliente HTTP envia HTTP *request message* (contendo a URL) para o *socket* da conexão TCP

3. Servidor HTTP recebe mensagem de pedido, forma *response* message contendo o objeto solicitado (someDepartment/home.index), envia mensagem para o socket

Tempo

HTTP não persistente

4. Servidor HTTP fecha conexão TCP.

5. Cliente HTTP recebe mensagem de resposta contendo o arquivo html, apresenta o conteúdo html. Analisando o arquivo html, encontra 10 objetos jpeg referenciados

Tempo

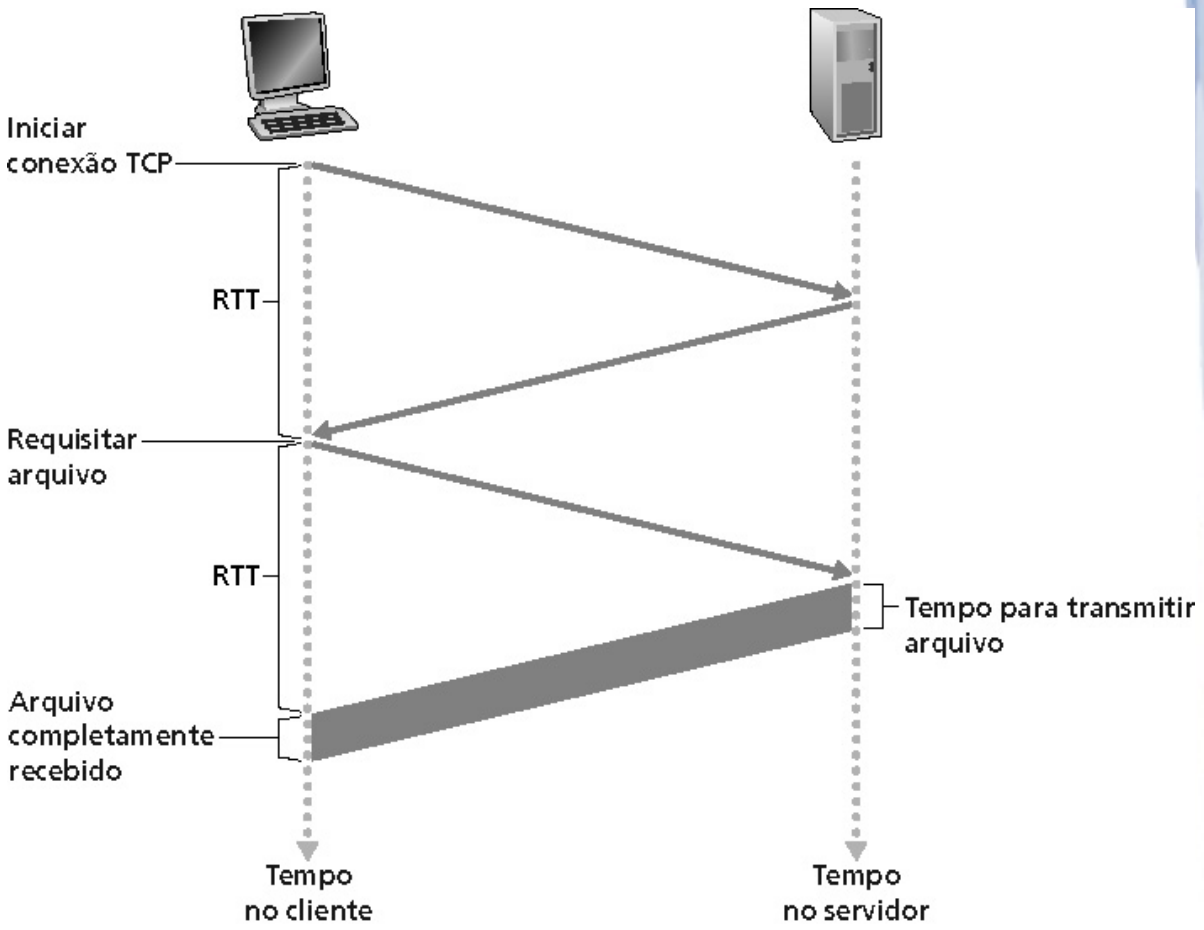
6. **Passos 1-5 são repetidos <u>para cada um</u> dos 10 objetos jpeg.**

Modelagem do tempo de resposta

Definição de RTT: tempo para enviar um pequeno pacote que vai do cliente para o servidor e retorna.

Tempo de resposta:

- Um RTT para iniciar a conexão TCP
- Um RTT para requisição HTTP e primeiros bytes da resposta HTTP para retorno
- Tempo de transmissão de arquivo



Total = 2 RTT+ tempo de transmissão

HTTP persistente

Características do HTTP não persistente:

- Requer 2 RTTs por objeto
- Sistema Operacional deve manipular e alocar recursos do hospedeiro para cada conexão TCP
  (Mas os *browsers* freqüentemente abrem conexões TCP paralelas para buscar objetos referenciados)

HTTP persistente

- Servidor deixa a conexão aberta após enviar uma resposta
- Mensagens HTTP subseqüentes entre o mesmo cliente/servidor são enviadas pela conexão

Persistente sem ***pipelining***:

- O cliente emite novas requisições apenas quando a resposta anterior for recebida
- Um RTT para cada objeto referenciado

Persistente com ***pipelining***:

- Padrão no HTTP/1.1
- O cliente envia requisições assim que encontra um objeto referenciado
- Tão curto quanto um RTT para todos os objetos referenciados

Mensagem HTTP request

- Dois tipos de mensagens HTTP: request, response
- HTTP request message:
  - ASCII (formato legível para humanos)

Linha de pedido (comandos GET, POST (para Formulários), HEAD (para depuração))

```
GET /somedir/page.html HTTP/1.0
```

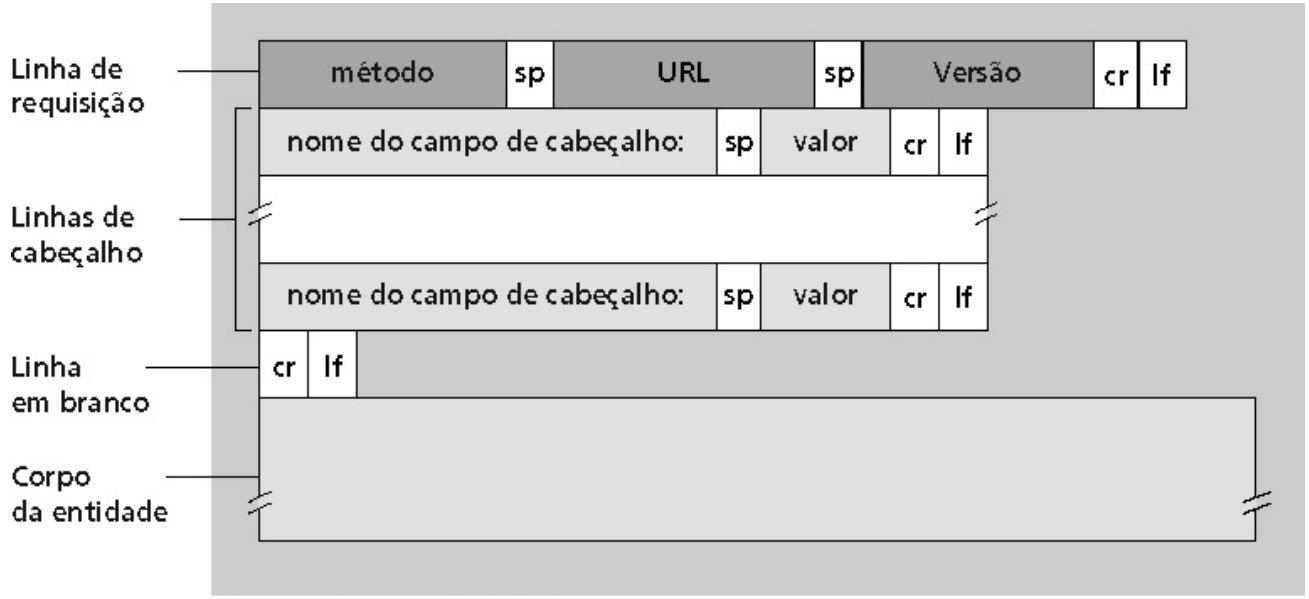
Linhas de cabeçalho

```
User-agent: Mozilla/4.0
Accept: text/html, image/gif,image/jpeg
Accept-language:fr
```

Carriage return, line feed indica fim da mensagem

(extra carriage return, line feed)

Mensagem HTTP request: formato geral



Obs.: cr = carriage return; lf = line feed

Entrada de formulário

Método Post:

- Página Web freqüentemente inclui entrada de formulário
- A entrada é enviada para o servidor no corpo da entidade

Método URL:

- Utiliza o método GET
- A entrada é enviada no campo de URL da linha de requisição:

www.somesite.com/animalsearch?monkeys&banana

Tipos de métodos

HTTP/1.0

- GET
- POST
- HEAD
  - Pede para o servidor deixar o objeto requisitado fora da resposta

HTTP/1.1

- GET, POST, HEAD
- PUT
  - Envia o arquivo no corpo da entidade para o caminho especificado no campo de URL
- DELETE
  - Apaga o arquivo especificado no campo de URL

Mensagem HTTP response

Linha de status (protocolo código de status frase de status)

Linhas de cabeçalho

Dados, ex.: arquivo html

```
HTTP/1.0 200 OK
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 …...

Content-Length: 6821
Content-Type: text/html

data data data data data ...
```

Códigos de *status* das respostas

Na primeira linha da mensagem de resposta servidor → cliente.
Alguns exemplos de códigos:

200 OK
- Requisição bem-sucedida, objeto requisitado a seguir nesta mensagem

301 Moved permanently
- Objeto requisitado foi movido, nova localização especificada a seguir nesta mensagem (Location:)

400 Bad request
- Mensagem de requisição não compreendida pelo servidor

404 Not Found
- Documento requisitado não encontrado neste servidor

505 HTTP version not supported

HTTP cliente: faça você mesmo!

1.Telnet para um servidor Web:

```
telnet cis.poly.edu 80
```

Abre conexão TCP para a porta 80 (porta *default* do servidor HTTP) em cis.poly.edu. Qualquer coisa digitada é enviada para a porta 80 em cis.poly.edu

2.Digite um pedido GET HTTP:

```
GET /~ross/ HTTP/1.1
host: cis.poly.edu
```

Digitando isso (tecle nova linha duas vezes), você envia este pedido HTTP GET mínimo (mas completo) ao servidor HTTP

3.Examine a mensagem de resposta enviada pelo servidor HTTP!

Estado usuário-servidor: *cookies*

A maioria dos grandes sítios Web utilizam *cookies*

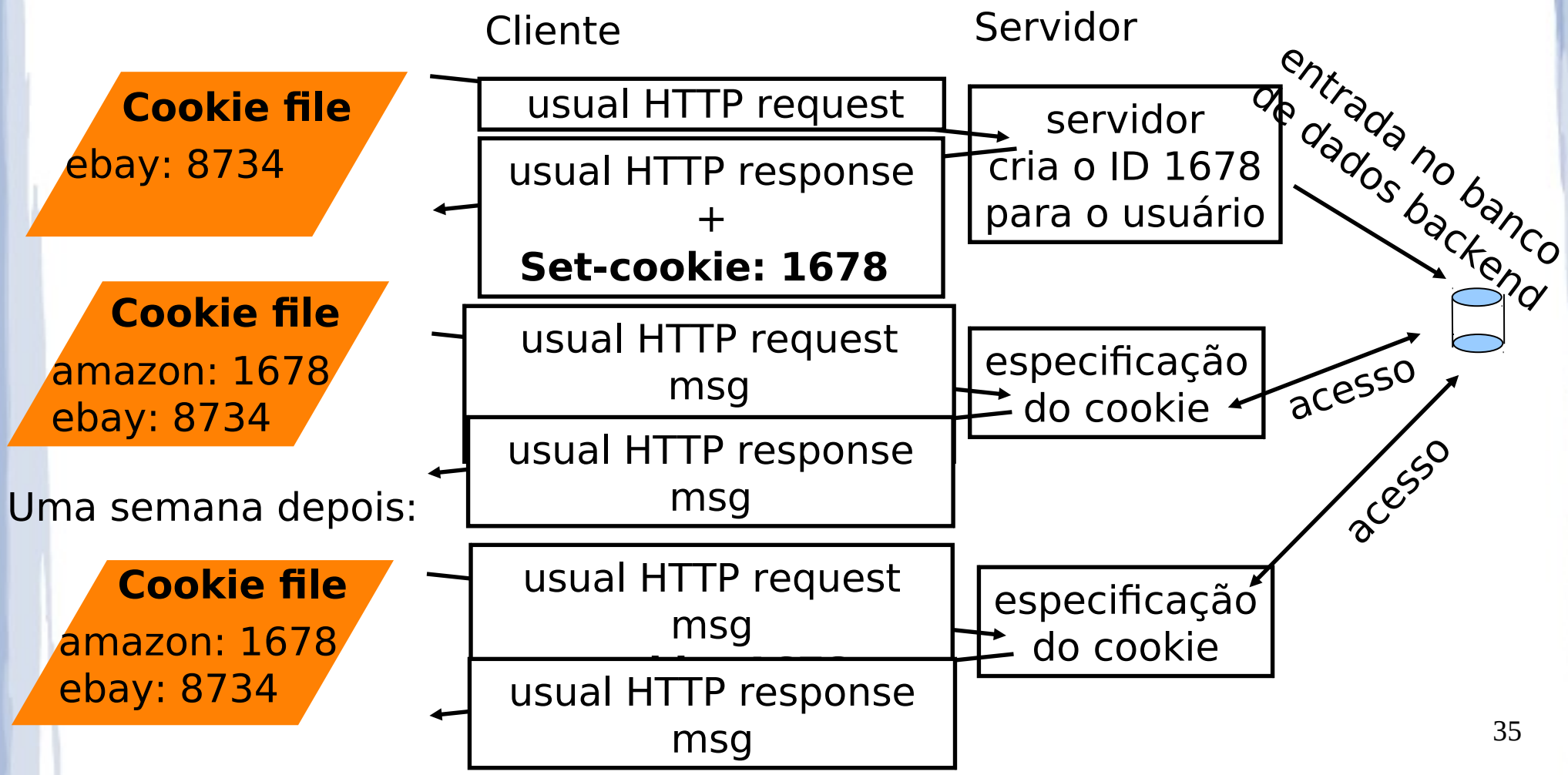
Quatro componentes:

1) Linha de cabeçalho do *cookie* na mensagem HTTP response
2) Linha de cabeçalho de cookie na mensagem HTTP request
3) Arquivo de *cookie* mantido no hospedeiro do usuário e manipulado pelo *browser* do usuário
4) Banco de dados ***backend*** no sítio Web

Exemplo:

- Susan acessa a Internet sempre do mesmo PC
- Ela visita um sítio específico de comércio eletrônico pela primeira vez
- Quando a requisição HTTP inicial chega ao sítio, este cria um identificador (ID) único e uma entrada no banco de dados ***backend*** para este ID

Cookies: mantendo “estado”



Cookies

O que os cookies podem trazer:

- Autorização
- Cartões de compra
- Recomendações
- Estado de sessão do usuário (Web e-mail)

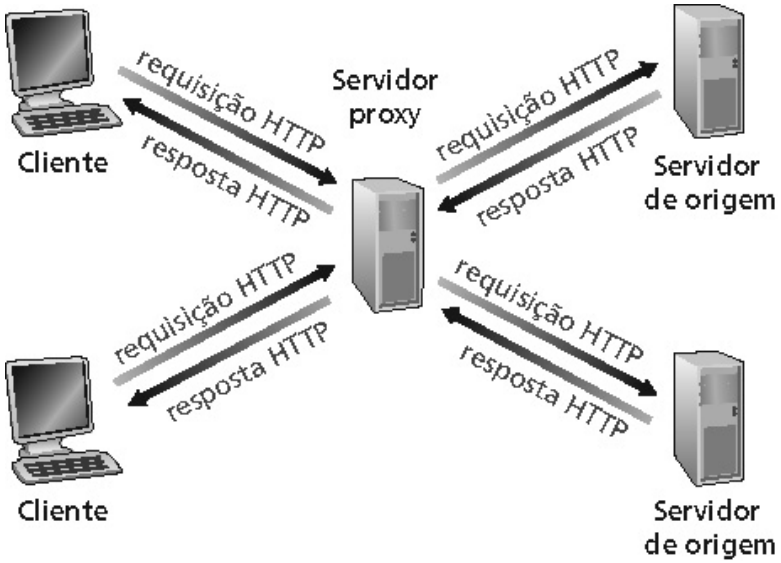
Cookies e privacidade:

- Cookies permitem que *sites* saibam muito sobre você
- Você pode fornecer nome e e-mail para os *sites*
- Mecanismos de busca usam redirecionamento e cookies para saberem mais sobre você
- Companhias de propaganda obtêm informações por meio dos *sites*

Web caches (proxy server)

Objetivo: atender o cliente sem envolver o servidor Web originador da informação

- Usuário configura o *browser*: acesso Web é feito por meio de um procurador (*proxy*).
- Cliente envia todos os pedidos HTTP para o *proxy*
  - Se o objeto existe na *cache* do procurador: o procurador retorna o objeto
  - Caso contrario, o procurador solicita o(s) objeto(s) do servidor original, e então envia o(s) objeto(s) ao cliente



Mais sobre Web caching

- A *cache* atua tanto no servidor como no cliente
- Tipicamente, a *cache* é instalada pelo ISP (universidade, companhia, ISP residencial)

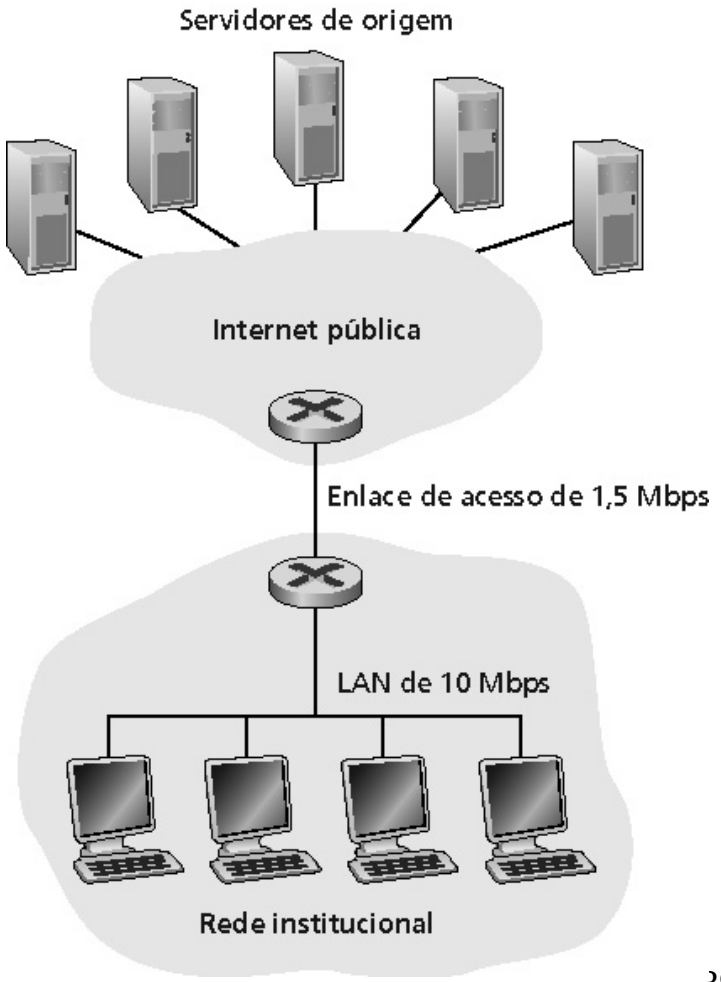
Por que Web caching?

- **Reduz o tempo de resposta** para a requisição do cliente.
- **Reduz o tráfego** em um enlace de acesso de uma instituição.
- A densidade de *caches* na Internet habilita os “fracos” provedores de conteúdo a efetivamente entregarem o conteúdo (mas fazendo *P2P file sharing*)

Exemplo de *caching*

Suponha:

- Tamanho médio objeto = **100.000 bits**
- Taxa média de requisições dos *browsers* da instituição para os servidores de origem = **15/s**
- Atraso total = atraso da Internet (roteador externo -> destino -> roteador externo) + atraso de acesso (entre roteador institucional e roteador externo) + atraso da LAN (rede local)
    - Exemplo (atraso Internet = 2 s): tempo total = 2 s + *x* ms + *y* ms



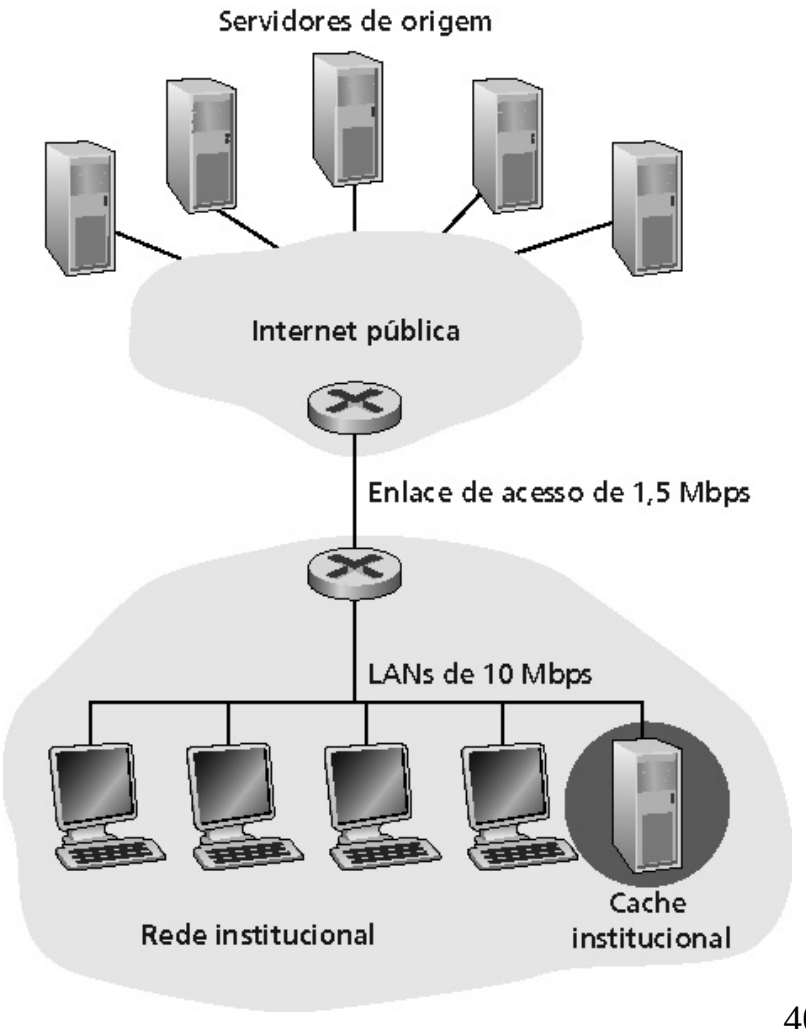
Conseqüências:

- Utilização da LAN = **15%**
- Utilização do link de acesso = **100%**

Exemplo de *caching*

Solução possível

- Aumentar a largura de banda do enlace de acesso, como, **<u>por exemplo, 10 Mbps</u>**
- Conseqüências
  - Utilização da LAN = 15%
  - Utilização do enlace de acesso = 15%
  - Freqüentemente é um ***upgrade*** caro
  - Para o exemplo anterior, tempo médio de acesso ainda em torno de 2 s (i.e., Tempo de Acesso a Internet é dominante!)



Exemplo de *caching*

Instalação do *proxy*

- Suponha que a taxa de acertos (*hit ratio*) seja ***0.4***

Conseqüência

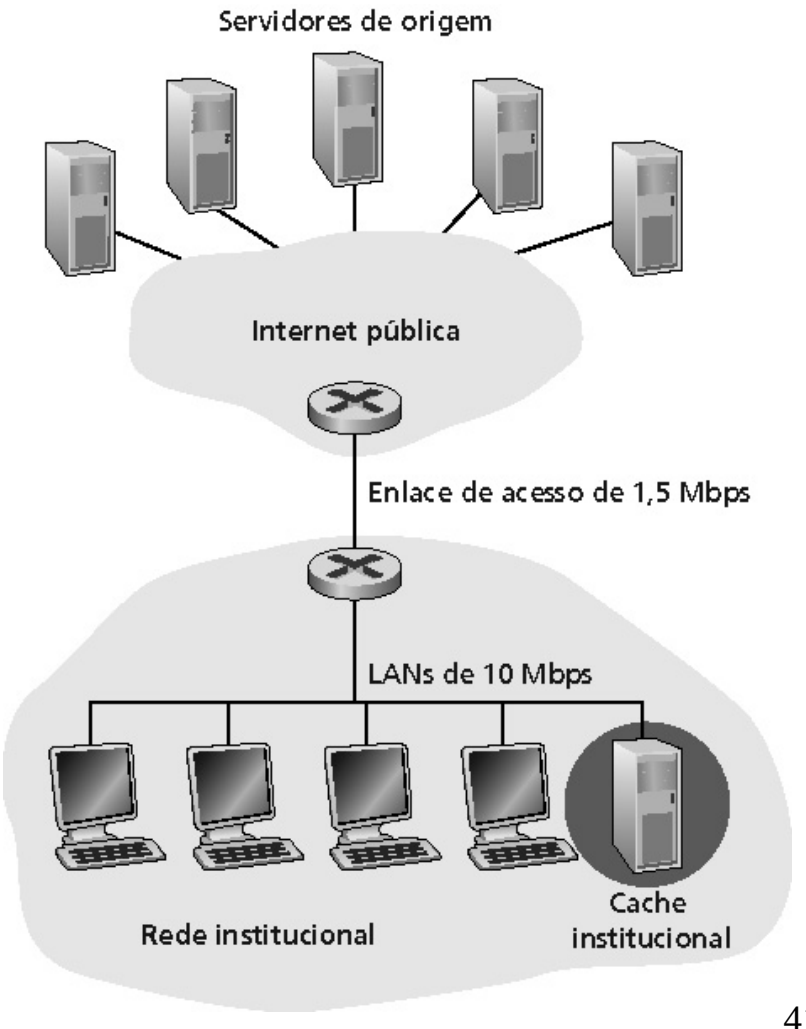
- ***40%*** das requisições serão satisfeitas quase que imediatamente pelo *proxy*
- ***60%*** das requisições serão satisfeitas pelo servidor de origem
- Utilização do enlace de acesso reduzida para 60%, resultando em atrasos insignificantes

(vamos assumir algo em torno de ***10 ms como tempo para obter resposta do web cache***):

tempo total médio =

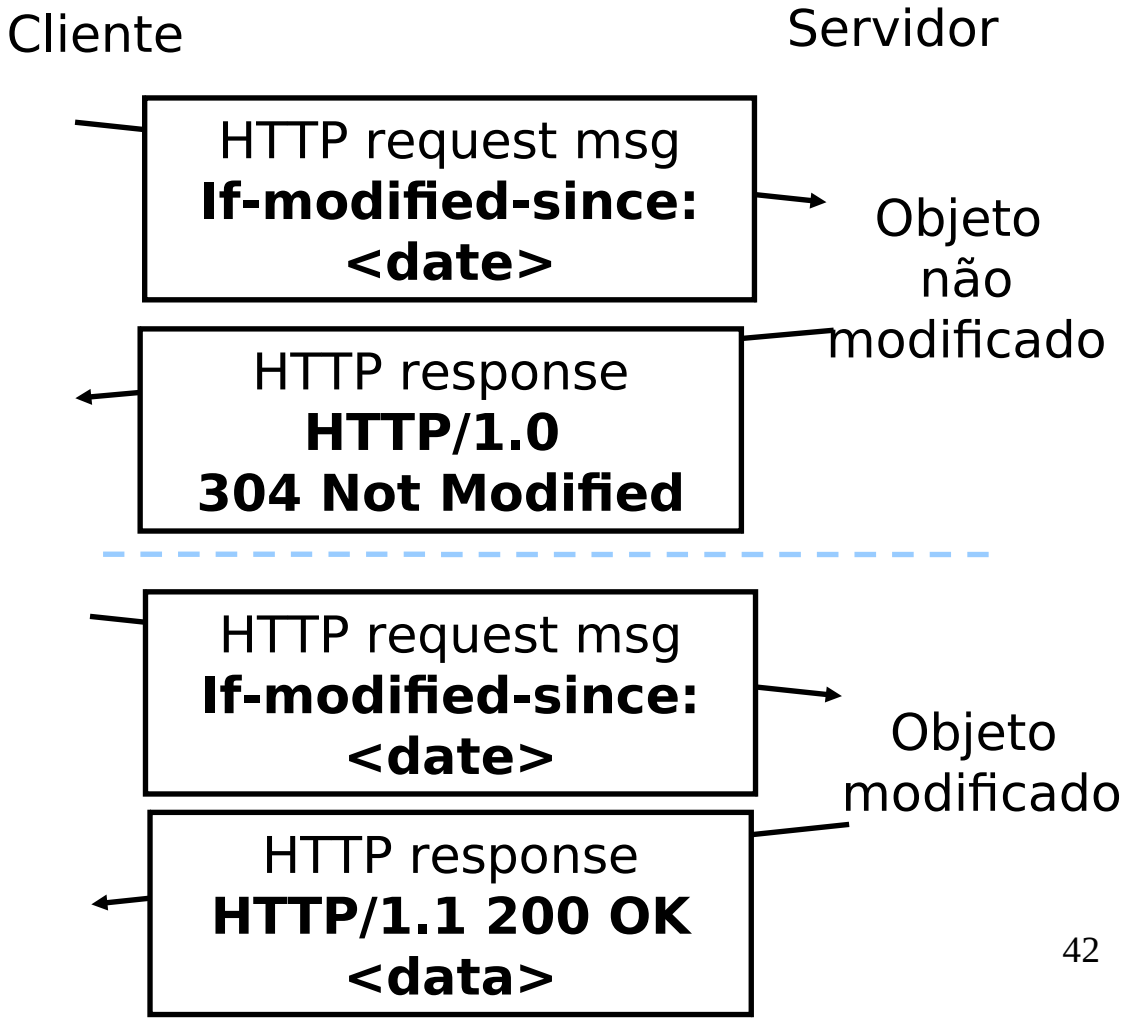
= 0.4 (0,01) + 0.6 (2 + 0,01)

= 1,2 s



GET condicional

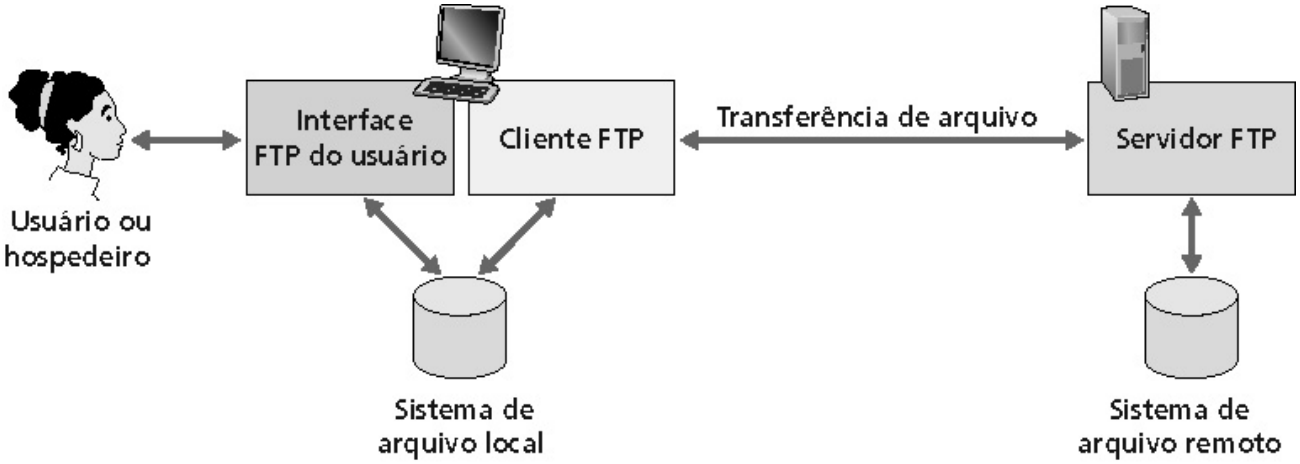
- Razão: não enviar objeto se a versão que o cliente já possui está atualizada.
- Cliente: especifica data da versão armazenada no pedido HTTP
  - **If-modified-since: <date>**
- Servidor: resposta não contém objeto se a cópia é atualizada: **HTTP/1.0 304 Not Modified**



Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

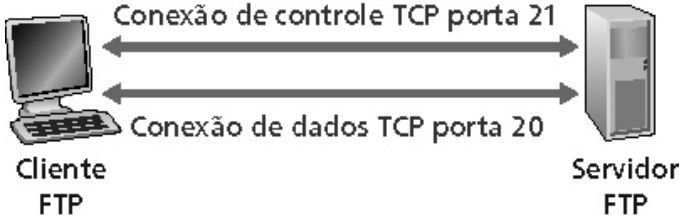
FTP: o protocolo de transferência de arquivos



- Transferência de arquivos de e para o computador remoto
- Modelo cliente servidor
  - **Cliente:** lado que inicia a transferência (seja de ou para o lado remoto)
  - **Servidor:** hospedeiro remoto
- FTP: RFC 959
- **FTP servidor: porta 21**

FTP: controle separado das conexões de dados

- Cliente FTP contata o servidor FTP na porta 21 especificando o TCP como protocolo de transporte
- Cliente obtém autorização pela conexão de controle
- Cliente procura o diretório remoto enviando comandos pela conexão de controle
- Quando o servidor recebe um comando para uma transferência de arquivo, ele abre uma conexão de dados TCP para o cliente
- Após a transferência de um arquivo, o servidor fecha a conexão
- Servidor abre uma segunda conexão de dados TCP para transferir outro arquivo
- Conexão de controle: "fora da banda" (out-of-band)
- Servidor FTP mantém "estado" sobre o usuário: diretório atual, autenticação anterior



FTP comandos, respostas

Exemplos de comandos:

- Envie um texto ASCII sobre canal de controle
- **USER *username***
- **PASS *password***
- **LIST** retorna listagem do arquivo no diretório atual
- **RETR filename** recupera (obtém) o arquivo
- **STOR filename** armazena o arquivo no hospedeiro remoto

Exemplos de códigos de retorno

- Código de *status* e frase (como no HTTP)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can’t open data connection**
- **452 Error writing file**

Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio eletrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

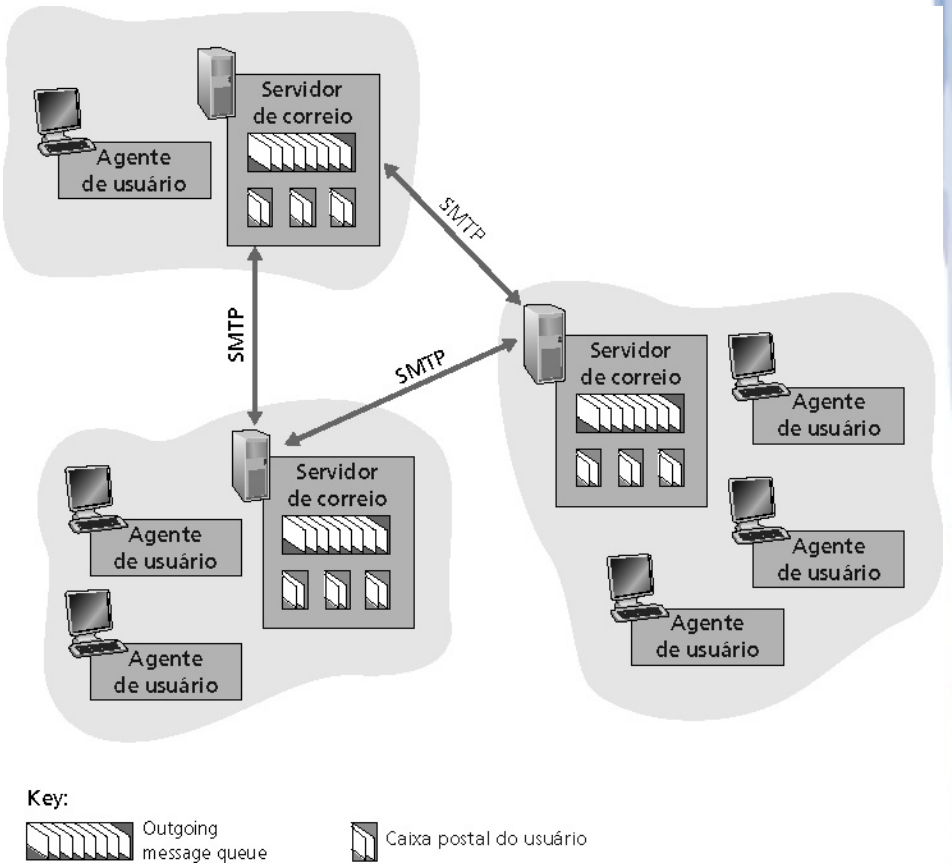
Correio eletrônico

Três componentes principais:

- Agentes de usuário
- Servidores de correio
- *Simple mail transfer protocol: SMTP*

Agente de usuário

- "leitor de correio"
- Composição, edição, leitura de mensagens de correio
- Ex.: Eudora, Outlook, elm, Netscape Messenger, Thunderbird
- Mensagens de entrada e de saída são armazenadas no servidor

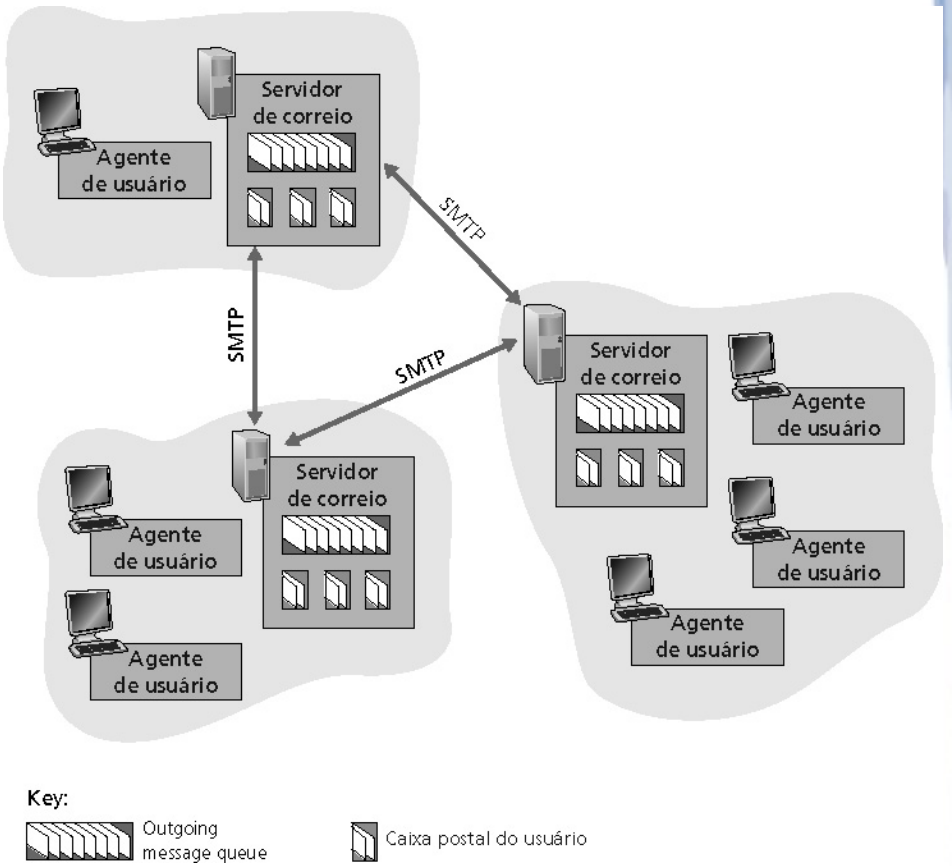


Correio eletrônico: servidores de correio

Servidores de correio

- Caixa postal contém mensagens que chegaram (ainda não lidas) para o usuário
- Fila de mensagens contém as mensagens de correio a serem enviadas

Protocolo SMTP permite aos servidores de correio trocarem mensagens entre si
- Cliente: servidor de correio que envia
- "servidor": servidor de correio que recebe

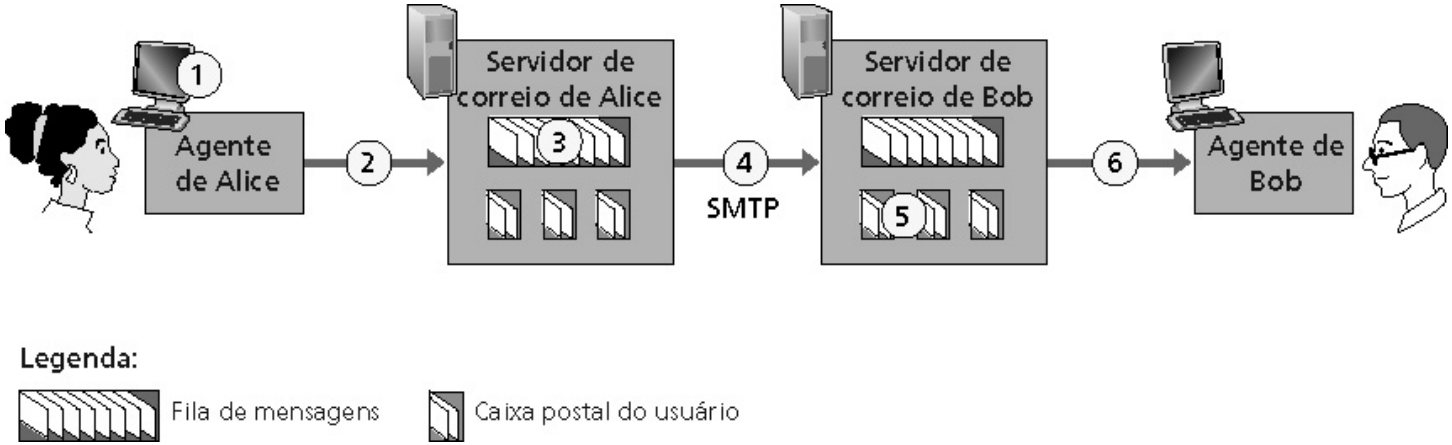


Correio eletrônico: SMTP [RFC 821]

- Usa TCP para transferência confiável de mensagens de correio do cliente ao servidor, porta 25
- Transferência direta: servidor que envia para o servidor que recebe
- Três fases de transferência
  - *Handshaking* (apresentação)
  - Transferência de mensagens
  - Fechamento
- Interação comando/resposta
  - Comandos: texto ASCII
  - Resposta: código de *status* e frase
- Mensagens devem ser formatadas em código ASCII de 7 bits

Cenário: Alice envia mensagem para Bob

1) Alice usa o agente de usuário (*User Agent*) para compor a mensagem "para" bob@someschool.edu
2) O agente de usuário dela envia a mensagem para o seu servidor de correio; a mensagem é colocada na fila de mensagens.
3) O lado cliente do SMTP abre uma conexão TCP com o servidor de correio do Bob.
4) O cliente SMTP envia a mensagem de Alice pela conexão TCP.
5) O servidor de correio de Bob coloca a mensagem na caixa de correio de Bob.
6) Bob invoca seu agente de usuário para ler a mensagem.



Exemplo de interação SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Tente o SMTP você mesmo

- **`telnet nome do servidor 25`**
- Veja resposta 220 do servidor
- Envie comandos HELO, MAIL FROM, RCPT TO, DATA, QUIT

  (a seqüência acima permite enviar um comando sem usar o agente de usuário do remetente)

SMTP: palavras finais

- SMTP usa conexões persistentes
- SMTP exige que as mensagens (cabeçalho e corpo) estejam em ASCII de 7 bits
- Servidor SMTP usa CRLF.CRLF para indicar o final da mensagem

Comparação com HTTP:

- HTTP: pull
- E-mail: push
- Ambos usam comandos e respostas em ASCII, interação comando/resposta e códigos de status
- HTTP: cada objeto encapsulado na sua própria mensagem de resposta
- SMTP: múltiplos objetos são enviados numa mensagem multiparte

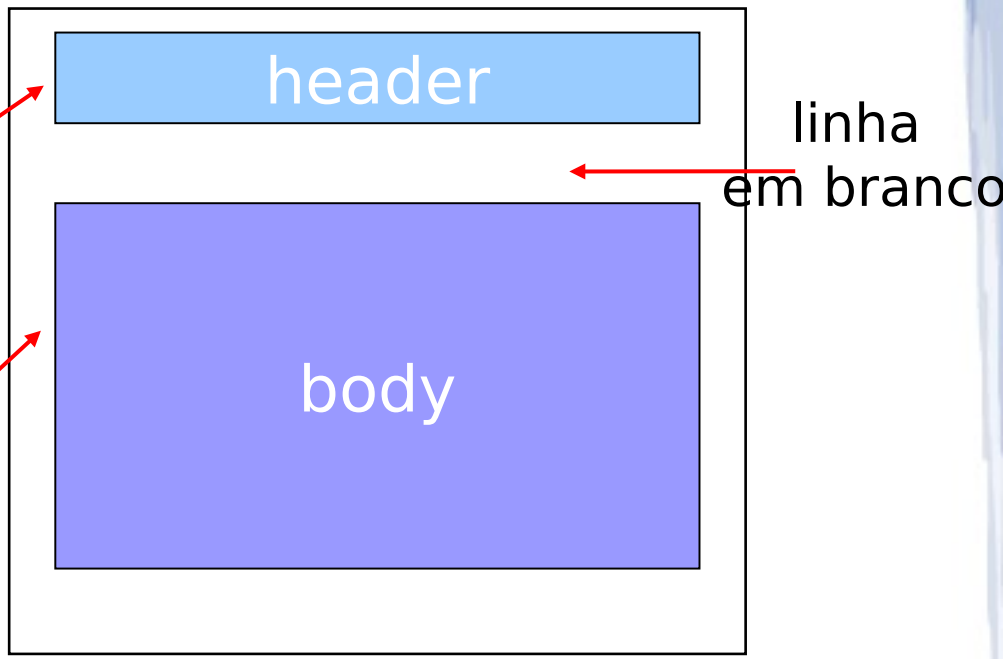
Formato da mensagem de correio

SMTP: protocolo para trocar mensagens de e-mail

RFC 822: padrão para mensagens do tipo texto:

- linhas de cabeçalho, ex.:
  - To:
  - From:
  - Subject:

  diferente *dos comandos HTTP*
- corpo
  - a "mensagem", ASCII somente com caracteres



Formato das mensagens: extensões multimídia

- MIME: *multimedia mail extension (*RFC 2045 e 2056)
- Linhas adicionais no cabeçalho declaram o tipo de conteúdo MIME

Versão do MIME

Método usado para codificar dados

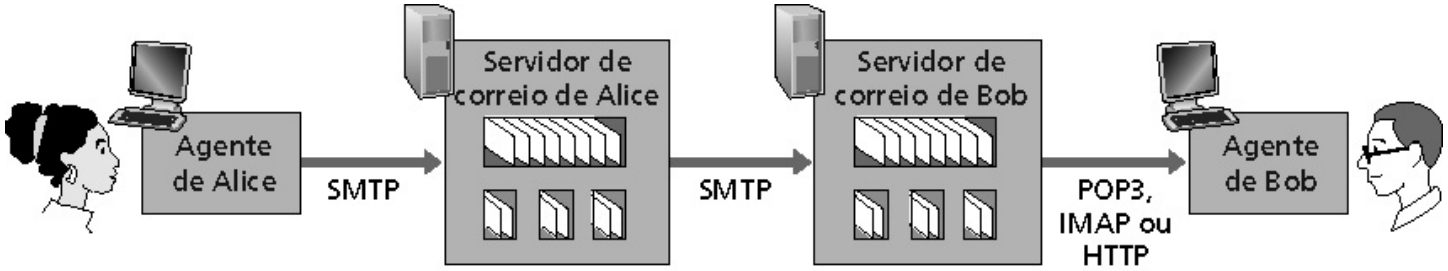
Dados multimídia tipo, subtipo, declaração de parâmetro

Dados codificados

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.........................
......base64 encoded data
```

Protocolos de acesso ao correio



- SMTP: entrega e armazena no servidor do destino
- Protocolo de acesso: recupera mensagens do servidor
  - POP: Post Office Protocol [RFC 1939]
    - Autorização (agente <-->servidor) e *download*
  - IMAP: Internet Mail Access Protocol [RFC 1730]
    - Maiores recursos (mais complexo)
    - Manipulação de mensagens armazenadas no servidor
  - HTTP: Hotmail , Yahoo! Mail, Gmail, etc.

Protocolo POP3

Fase de autorização

- comandos do cliente:
  - **user:** declara nome do usuário
  - **pass:** *password*

respostas do servidor

- **+OK**
- **-ERR**

Fase de transação,

- **list:** lista mensagens e tamanhos
- **retr:** recupera mensagem pelo número
- **dele:** apaga
- **quit**

```
S: +OK POP3 server ready
C: user alice
S: +OK
C: pass hungry
S: +OK user successfully
logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (continuação) e IMAP

Mais sobre POP3

- O exemplo anterior usa o modo *download-and-delete*
- Bob não pode reler o e-mail se ele trocar o cliente
- *download-and-keep*: cópias das mensagens em clientes diferentes
- POP3 é *stateless* através das sessões

IMAP

- Mantém todas as mensagens em um lugar: o servidor
- Permite que o usuário organize as mensagens em pastas
- IMAP mantém o estado do usuário através das sessões:
  - Nomes das pastas e mapeamentos entre os IDs da mensagem e o nome da pasta

Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

DNS: *Dominain Name System*

Pessoas: muitos identificadores:
- RG, nome, passaporte, CPF

Internet hospedeiros, roteadores:
- Endereços IP (32 bits) - usados para endereçar datagramas
- “nome”, ex.: www.dsc.ufcg.edu.br - usados por humanos

P.: Como relacionar nomes com endereços IP?

Domain Name System:

- **Base de dados distribuída** implementada numa hierarquia de muitos **servidores de nomes**
- **Protocolo de camada de aplicação** hospedeiro, roteadores se comunicam com servidores de nomes para **resolver** nomes (tradução nome/endereço)
  - Nota: função interna da Internet, implementada como protocolo da camada de aplicação
  - Complexidade na “borda” da rede

DNS

DNS services

- Nome do hospedeiro para tradução de endereço IP
- Hospedeiro *aliasing* (apelido)
  - Nomes canônicos e *alias*

    mail server aliasing

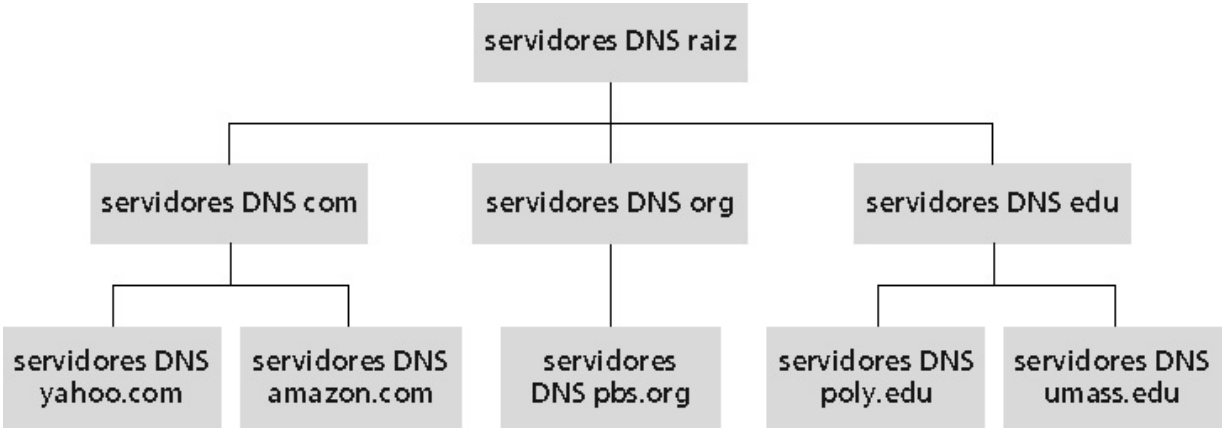
    distribuição de carga
  - Servidores Web replicados: estabelece o endereço IP para um nome canônico

Por que não centralizar o DNS?

- Ponto único de falha
- Volume de tráfego
- Base centralizada de dados distante
- Manutenção

Não é escalável!

Base de dados distribuída, hierárquica



Cliente quer o IP para www.amazon.com; 1ª aprox.:

- Cliente consulta um servidor de raiz para encontrar o servidor **DNS .com**
- Cliente consulta o servidor DNS com para obter o servidor **DNS amazon.com**
- Cliente consulta o servidor DNS amazon.com para obter o endereço IP para **www.amazon.com**

DNS: servidores de nomes raiz

- São contatados pelos servidores de nomes locais que não podem resolver um nome
- Servidores de nomes raiz:
  - Buscam servidores de nomes autorizados se o mapeamento do nome não for conhecido
  - Conseguem o mapeamento
  - Retornam o mapeamento para o servidor de nomes local

Existem 13 servidores de nomes raiz no mundo

Servidores TLD e servidores com autoridade

Servidores ***Top-Level Domain*** (TLD): responsáveis pelos domínios com, org, net, edu etc e todos os domínios **top-level** nacionais uk, fr, ca, jp, br

- Network Solutions mantém servidores para o TLD “com”
- Educause para o TLD “edu”

Servidores DNS com autoridade: servidores DNS de organizações, provêem nome de hospedeiro autorizado para mapeamentos IP para servidores de organizações (ex.: Web e mail).

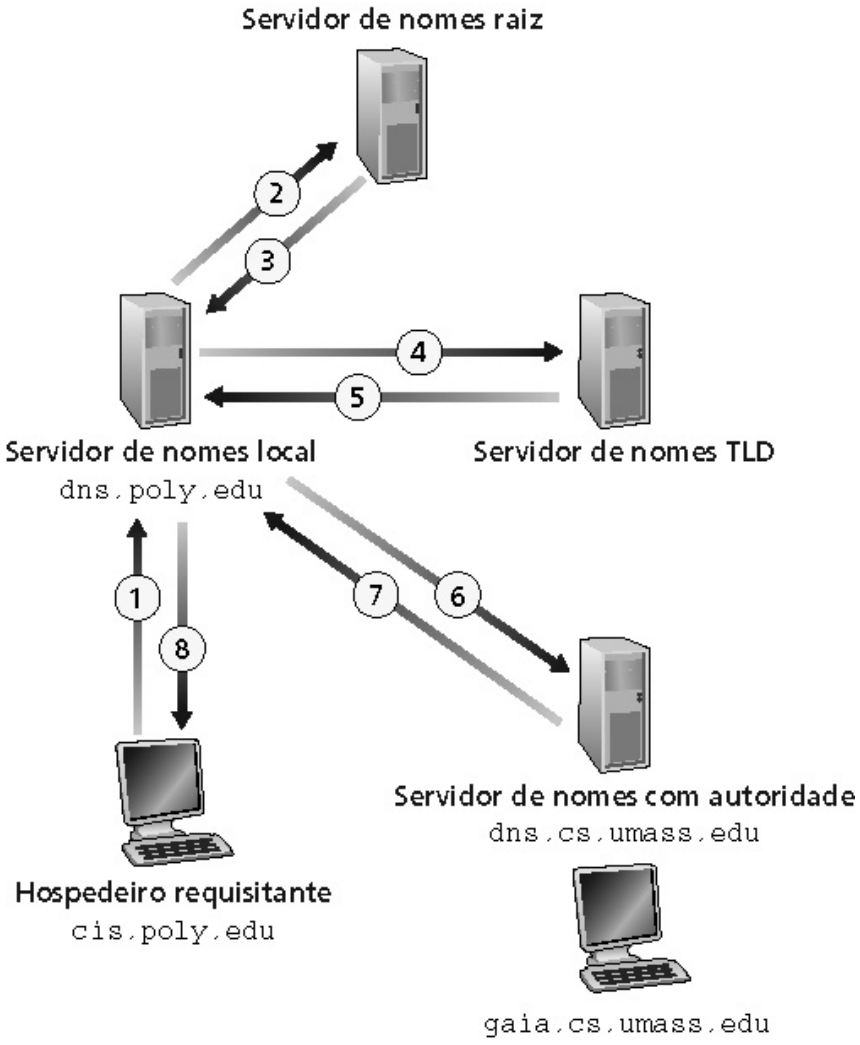
- Podem ser mantidos pela própria organização ou terceirizado em um provedor de serviços

Servidor de nomes local

- Não pertence estritamente a uma hierarquia
- Cada ISP (ISP residencial, companhia, universidade) possui um
  - Também chamado de “servidor de nomes *default*”
- Quando um hospedeiro faz uma pergunta a um DNS, a pergunta é enviada para seu servidor DNS local
  - Age como um procurador (*proxy*), encaminhando as perguntas para dentro da hierarquia

Exemplo

- O hospedeiro em cis.poly.edu quer o endereço IP para gaia.cs.umass.edu



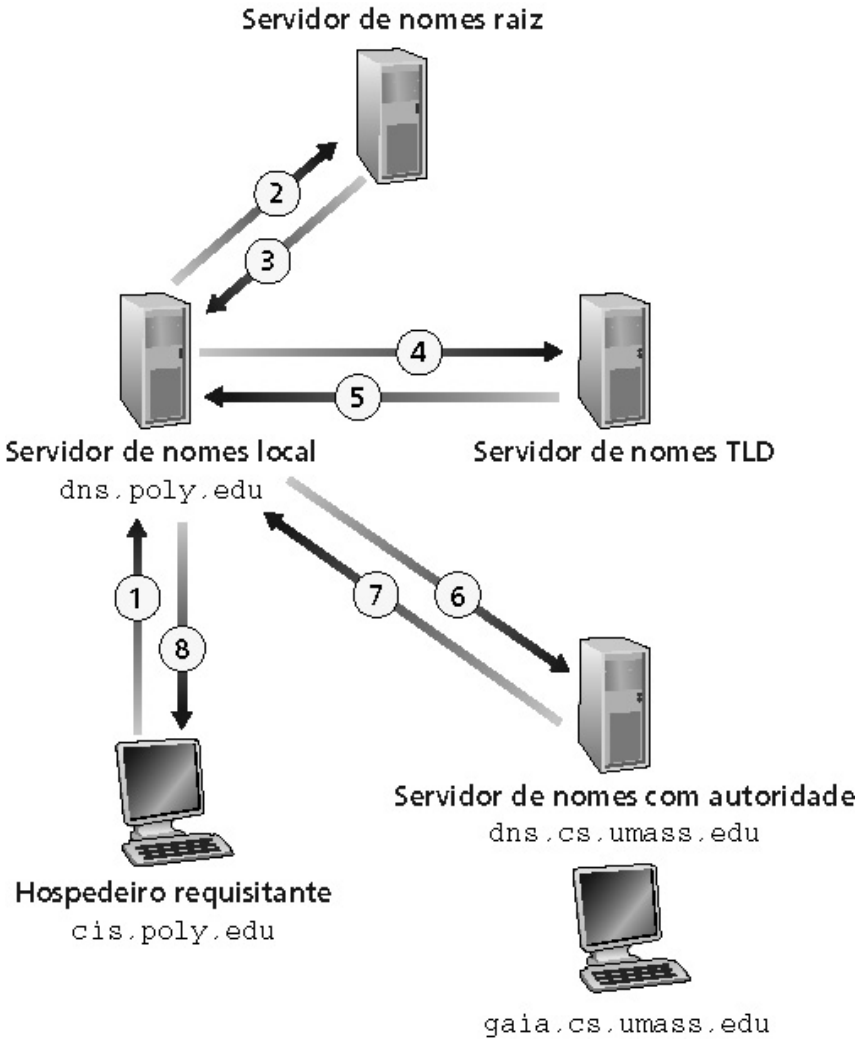
Consultas recursivas

Consulta recursiva:

- Transfere a tarefa de resolução do nome para o servidor de nomes consultado
- Carga pesada?

Consulta encadeada:

- Servidor contatado responde com o nome de outro servidor de nomes para contato
- **“eu não sei isto, mas pergunte a este servidor”**



DNS: armazenando e atualizando registros

Uma vez que um servidor de nomes aprende um mapeamento, ele armazena o mapeamento num registro do tipo ***cache***

- Registro do *cache* tornam-se obsoletos (desaparecem) depois de um certo tempo
- Servidores TLD são tipicamente armazenados em *cache* nos servidores de nome locais

Mecanismos de atualização e notificação estão sendo projetados pelo IETF

- RFC 2136
- http://www.ietf.org/html.charters/dnsind-charter.html

Registros do DNS

DNS: base de dados distribuída que armazena **registros de recursos** (RR)

formato dos RR: **(name, value, type,ttl)**

- Type = A
  - **name** é o nome do computador
  - **value** é o endereço IP
- Type = NS
  - **name** é um domínio (ex.: foo.com)
  - **value** é o endereço IP do servidor de nomes autorizados para este domínio
- Type = CNAME
  - **name** é um “apelido” para algum nome “canônico” (o nome real) www.ibm.com é realmente www.ibm.com.cs186.net
  - **value** é o nome canônico
- Type = MX
  - **value** é o nome do servidor de correio associado com **name**

DNS: protocolo e mensagem

Protocolo DNS: mensagem de **consulta** e resposta , ambas com o mesmo **formato de mensagem**

Cabeçalho da msg

- Identificação: número de 16 bits para consulta, resposta usa o mesmo número
- Flags:
  - Consulta ou resposta
  - Recursão desejada
  - Recursão disponível
  - Resposta é autorizada

Camada de aplicação

DNS: protocolo e mensagens

Camada de aplicação

- Exemplo: empresa recém-criada “Network Utopia”
- Registrar o nome networkuptopia.com num “registrar” (ex.: Network Solutions)
  - É necessário fornecer ao registrar os nomes e endereços IP do seu servidor de nomes autorizados (primário e secundário)
  - *Registrar* insere dois RRs no servidor TLD do domínio com:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- No servidor autorizado, inserir um registro Tipo A para www.networkuptopia.com e um registro Tipo MX para networkutopia.com
- Como as pessoas obtêm o endereço IP do seu Web site?

Inserindo registros no DNS

Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

Compartilhamento de arquivos entre-pares (*Peer-to-Peer*, P2P)

Exemplo

- Alice executa a aplicação cliente P2P em seu *notebook*
- Intermitentemente, conecta-se à Internet; obtém novos endereços IP para cada conexão
- pede por “Hey Jude”
- a aplicação exibe outros pares que possuem uma cópia de *Hey Jude*.
- Alice escolhe um dos pares, Bob.
- o arquivo é copiado do PC de Bob para o *notebook* de Alice: HTTP
- enquanto Alice faz o download, outros usuários fazem upload de Alice.
- o par de Alice é tanto um cliente Web como um servidor Web transiente.

Todos os pares são servidores = altamente escaláveis!

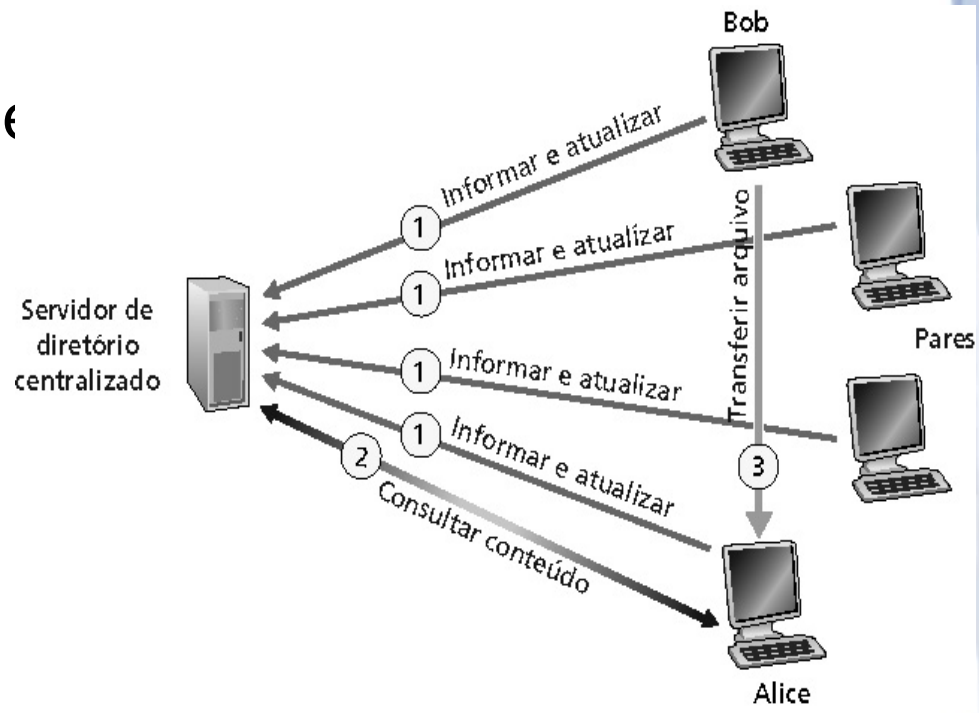
P2P: diretório centralizado

Projeto original “Napster”

- Quando um par se conecta, ele informa ao servidor central:
  - Endereço IP
  - Conteúdo

Exemplo de consulta:

- Alice procura por “Hey Jude”
- Servidor retorna lista de pares com o conteúdo desejado (e.g., máquina de Bob)
- Alice requisita o arquivo de Bob



P2P: problemas com diretório centralizado

- Ponto único de falhas
- Gargalo de desempenho
- Fácil identificar infrações de *copyright!?*

Transferência de arquivo é descentralizada, mas a localização de conteúdo é altamente centralizado

Query flooding: *Gnutella*

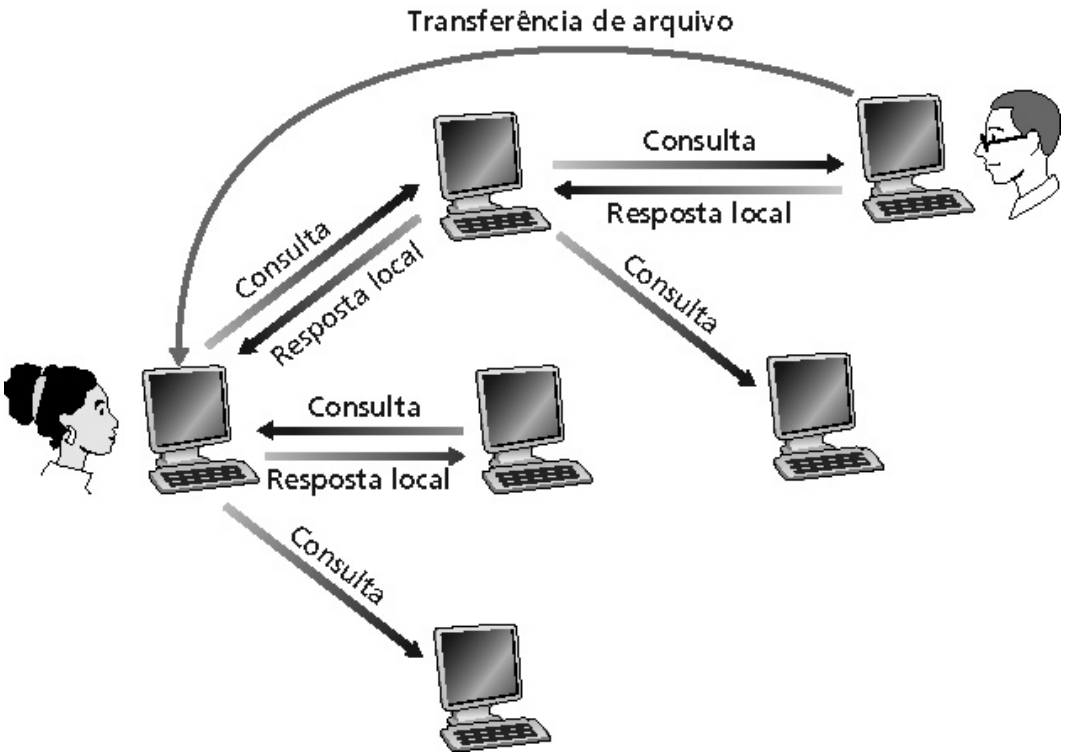
- Totalmente distribuído
  - Sem servidor central
- Protocolo de domínio público
- Muitos clientes Gnutella implementando o protocolo

Rede de cobertura: gráfico (topologia virtual)

- Aresta entre os pares X e Y indica que há uma conexão TCP entre os dois
- Todos os pares ativos e arestas estão na rede de sobreposição *(overlay network)*
- aresta não é um enlace físico: é virtual!
- Um determinado par será tipicamente conectado a <10 vizinhos na rede de sobreposição

Gnutella: protocolo

- Mensagem de consulta (*query*) é enviada pelas conexões TCP existentes
- Os pares encaminham a mensagem de consulta
- *QueryHit* (encontro) é enviado pelo caminho reverso



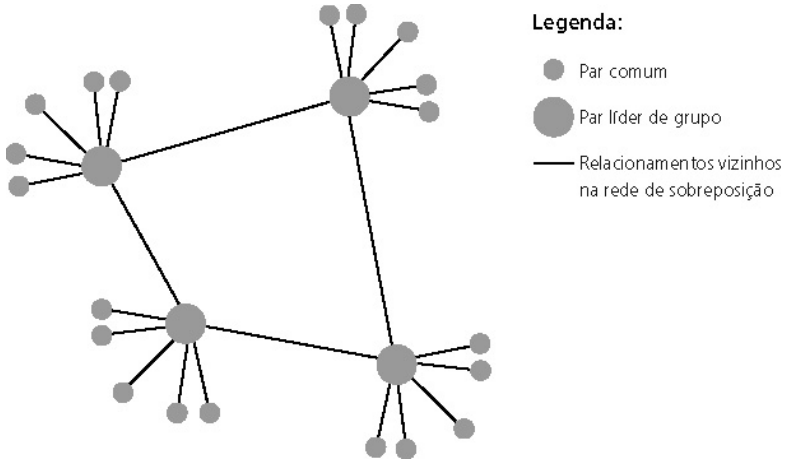
Escalabilidade: *flooding* de alcance limitado; limitar a inundação até uma certa distância (*expanding ring search*)

Gnutella: conectando pares

1. Para conectar o par X, ele precisa encontrar algum outro par na rede Gnutella: utiliza a lista de pares candidatos
2. X sequencialmente tenta fazer conexão TCP com os pares da lista até estabelecer conexão com, por exemplo, o par Y
3. X envia mensagem de Ping para Y; Y encaminha a mensagem de Ping.
4. Todos os pares que recebem a mensagem de Ping respondem com mensagens de Pong.
5. X recebe várias mensagens de Pong. Ele pode então estabelecer conexões TCP adicionais.

Explorando heterogeneidade: KaZaA

- Cada par é ou um líder de grupo ou está associado a um líder de grupo
  - Conexão TCP entre o par e seu líder de grupo
  - Conexões TCP entre alguns pares de líderes de grupo
- O líder de grupo acompanha o conteúdo em todos os seus "discípulos"



KaZaA

- Baseado no Protocolo *FastTrack*
- Utilizando engenharia reversa, a parte do protocolo entre cliente e super-nó foi descoberta
- A parte do protocolo falada entre os super nós ainda é desconhecida!
- Os mesmos desenvolvedores do KaZaA posteriormente criaram o **Skype**

KaZaA

- Cada arquivo possui um *hash* e um descritor
- O cliente envia a consulta de palavra-chave para o seu líder de grupo
- O líder de grupo responde com os encontros:
  - Para cada encontro: metadata, *hash*, endereço IP
- Se o líder de grupo encaminha a consulta para outros líderes de grupo, eles respondem com os encontros
- O cliente então seleciona os arquivos para *download*
  - Requisições HTTP usando hash como identificador são enviadas aos pares que contêm o arquivo desejado

Artifícios do KaZaA

- Limitações em *uploads* simultâneos
- Requisita enfileiramento
- Incentiva prioridades
- Realiza *downloads* em paralelo

Camada de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico

  SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

Programação de sockets

Objetivo: aprender a construir aplicações cliente-servidor que se comunicam usando sockets

Socket API

- Introduzida no UNIX BSD4.1 em 1981
- Explicitamente criados, usados e liberados pelas aplicações
- Paradigma cliente-servidor
- Dois tipos de serviço de transporte via *socket* API:
  - Datagrama não confiável (UDP)
  - Confiável, orientado a cadeias de bytes (TCP)

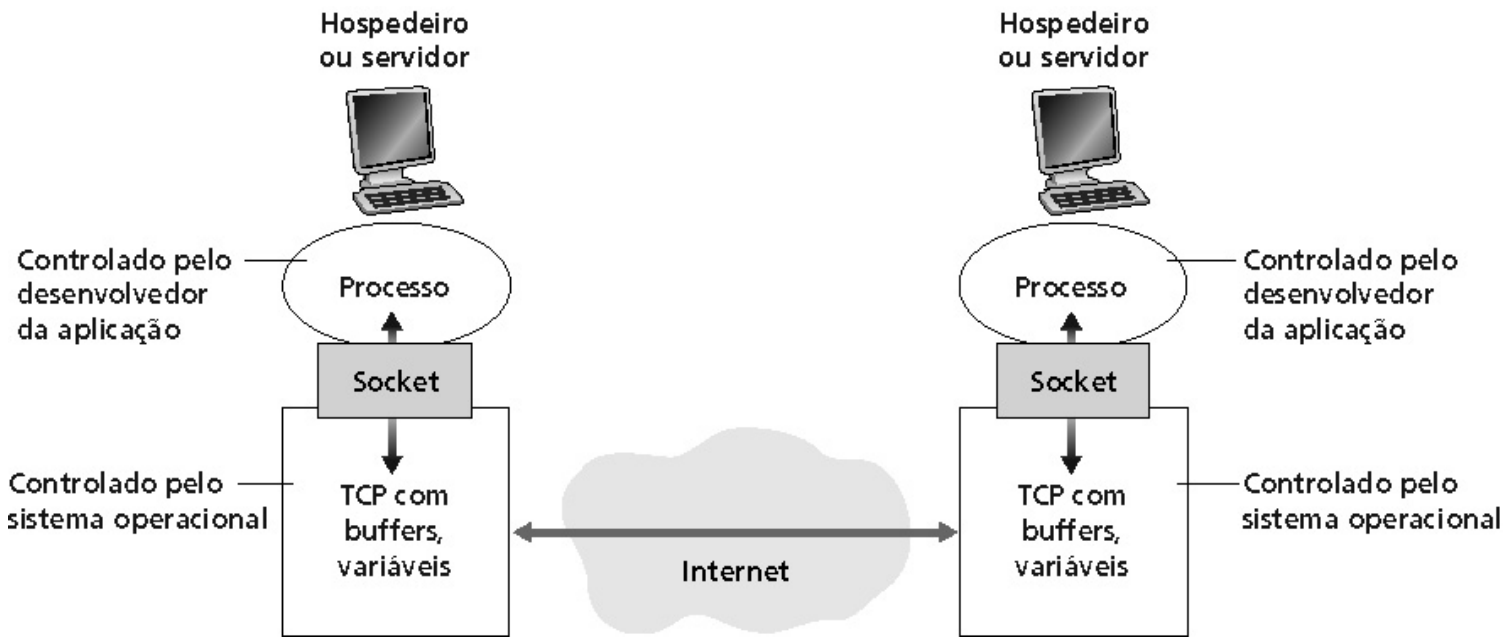
SOCKET

Uma interface **local,** criada por aplicações, **controlada pelo OS** (uma "porta") na qual os processos de aplicação podem tanto enviar quanto receber mensagens de e para outro processo de aplicação (local ou remoto)

Programação de sockets com TCP

Socket: uma porta entre o processo de aplicação e o protocolo de transporte fim-a-fim (UDP ou TCP)

Serviço TCP: transferência confiável de **bytes** de um processo para outro



Programação de sockets **com TCP**

Cliente deve contatar o servidor
- Processo servidor já deve estar em execução
- Servidor deve ter criado socket (porta) que aceita o contato do cliente

Cliente contata o servidor
- Criando um socket TCP local
- Especificando endereço IP e número da porta do processo servidor
- Quando o cliente cria o socket: cliente TCP estabelece conexão com o TCP do servidor

Quando contatado pelo cliente, o TCP do servidor cria um novo socket para o processo servidor comunicar-se com o cliente
- Permite ao servidor conversar com múltiplos clientes
- Número da porta de origem é utilizada para distinguir o cliente (mais sobre isso quando abordarmos a camada de Transporte)

Ponto de vista da aplicação

TCP fornece a transferência confiável e ordenada entre o cliente e o servidor

Jargão: *stream*

- Um *stream* é uma seqüência de caracteres que fluem para dentro ou para fora de um processo
  - Um *stream* de entrada é associada a alguma fonte de entrada para o processo. Ex.: teclado ou *socket*
  - Um *stream* de saída é associada a uma fonte de saída. Ex.: monitor ou *socket*

Programação de sockets **com TCP**

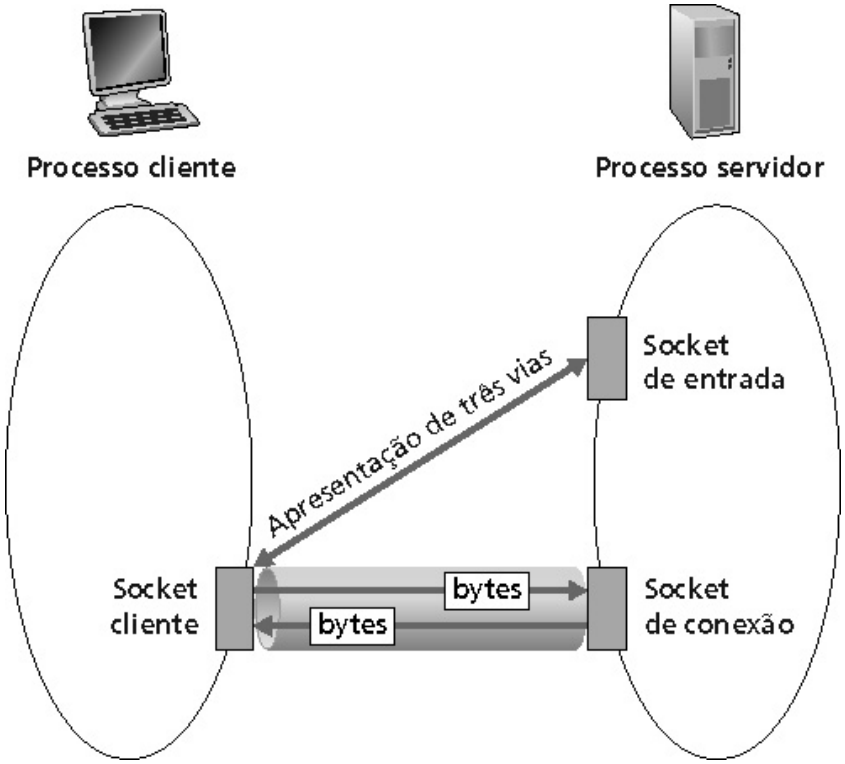
Exemplo de aplicação cliente-servidor:

1) Cliente lê linha da entrada-padrão do sistema (**inFromUser** stream), envia para o servidor via socket (**outToServer** stream)

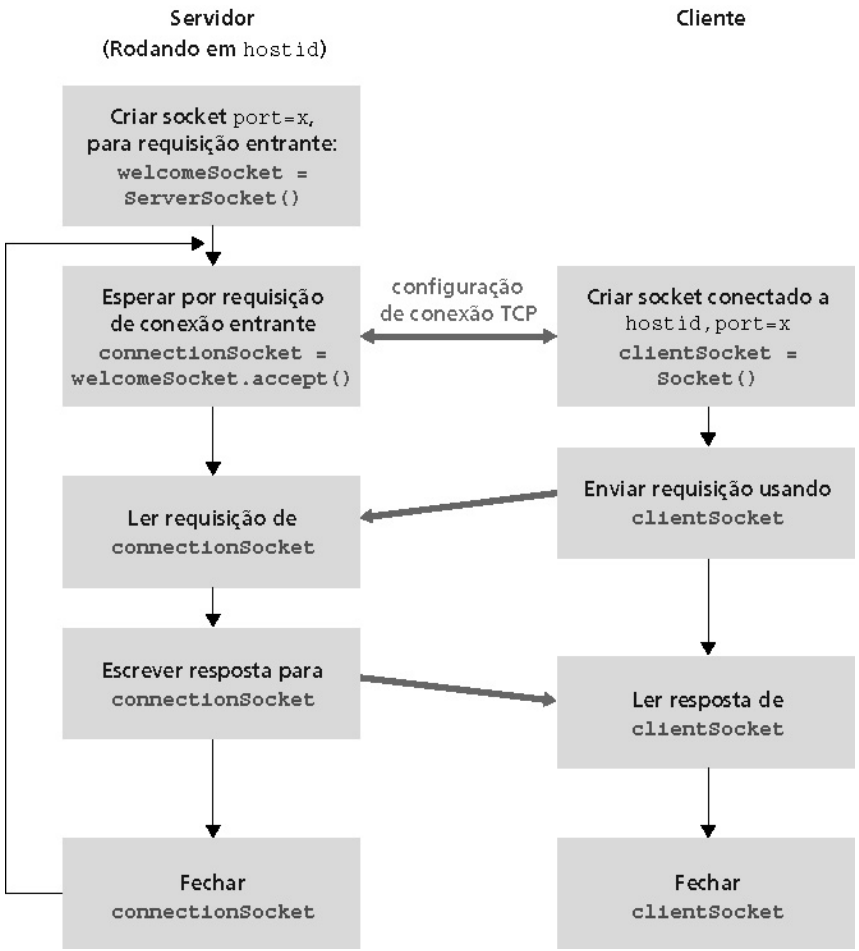
2) Servidor lê linha do socket

3) Servidor converte linha para letras maiúsculas e envia de volta ao cliente

4) Cliente lê a linha modificada através do (**inFromServer** stream)



Interação cliente-servidor TCP



Exemplo: cliente Java (TCP)

```
import java.io.*;
import java.net.*;
class TCPClient {

    public static void main(String argv[]) throws Exception
    {
        String sentence;
        String modifiedSentence;

        BufferedReader inFromUser =
          new BufferedReader(new InputStreamReader(System.in));

        Socket clientSocket = new Socket("hostname", 6789);

        DataOutputStream outToServer =
          new DataOutputStream(clientSocket.getOutputStream());
```

Cria stream de entrada

Cria socket cliente, conecta ao servidor

Cria stream de saída ligado ao socket

Exemplo: cliente Java (TCP)

Cria stream de entrada ligado ao socket

Envia linha para o servidor

Lê linha do servidor

```
BufferedReader inFromServer =
    new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));

sentence = inFromUser.readLine();

outToServer.writeBytes(sentence + '\n');

modifiedSentence = inFromServer.readLine();

System.out.println("FROM SERVER: " + modifiedSentence

clientSocket.close();

    }
}
```

Exemplo: servidor Java (TCP)

```
import java.io.*;
import java.net.*;

class TCPServer {

  public static void main(String argv[]) throws Exception
    {
      String clientSentence;
      String capitalizedSentence;

      ServerSocket welcomeSocket = new ServerSocket(6789);

      while(true) {

        Socket connectionSocket = welcomeSocket.accept();

        BufferedReader inFromClient =
          new BufferedReader(new
          InputStreamReader(connectionSocket.getInputStrean
```

Cria socket de aceitação na porta 6789

Espera, no socket de aceitação, por contato do cliente

Cria stream de entrada ligado ao socket

Exemplo: servidor Java (TCP)

Cria stream de saída ligado ao socket

```
DataOutputStream  outToClient =
  new DataOutputStream(connectionSocket.getOutputStream(
```

Lê linha do socket

```
clientSentence = inFromClient.readLine();

capitalizedSentence = clientSentence.toUpperCase() + '\n';
```

Escreve linha para o socket

```
outToClient.writeBytes(capitalizedSentence);
        }
    }
}
```

Fim do while loop, retorne e espere por outra conexão do cliente

Camadas de aplicação

- 2.1 Princípios de aplicações de rede
- 2.2 Web e HTTP
- 2.3 FTP
- 2.4 Correio electrônico
  - SMTP, POP3, IMAP
- 2.5 DNS
- 2.6 Compartilhamento de arquivos P2P
- 2.7 Programação de socket com TCP
- 2.8 Programação de socket com UDP
- 2.9 Construindo um servidor Web

Programação de sockets **com UDP**

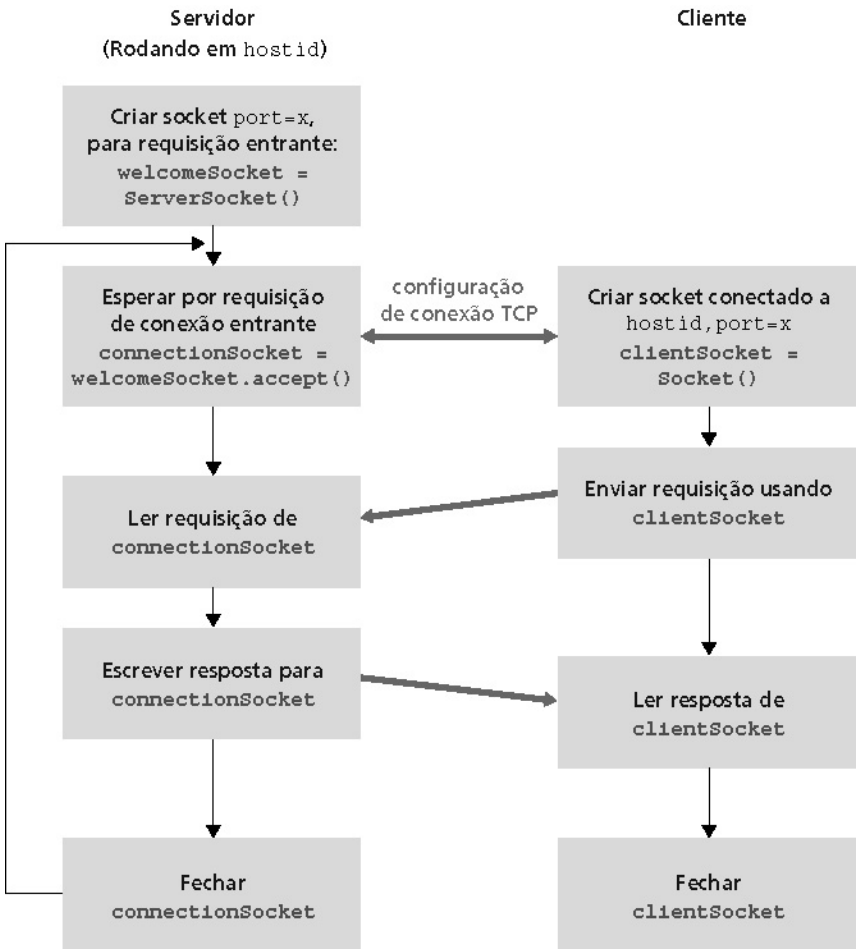
UDP: não há conexão entre o cliente e o servidor

- Não existe apresentação
- Transmissor envia explicitamente endereço IP e porta de destino em cada mensagem
- Servidor deve extrair o endereço IP e porta do transmissor de cada datagrama recebido
- Dados transmitidos podem ser recebidos fora de ordem ou perdidos

Ponto de vista da aplicação:

UDP fornece a transferência não confiável de grupos de bytes
(datagramas) entre o cliente e o servidor

Interação cliente-servidor: UDP



Exemplo: cliente Java (UDP)

Cria stream de entrada

Cria socket cliente

Translada nome do hospedeiro para endereço IP usando DNS

```
import java.io.*;
import java.net.*;

class UDPClient {
    public static void main(String args[]) throws Exception
    {

      BufferedReader inFromUser =
        new BufferedReader(new
InputStreamReader(System.in));

      DatagramSocket clientSocket = new DatagramSocket();

      InetAddress IPAddress =
InetAddress.getByName("hostname");

      byte[] sendData = new byte[1024];
      byte[] receiveData = new byte[1024];

      String sentence = inFromUser.readLine();
      sendData = sentence.getBytes();
```

Exemplo: cliente Java (UDP)

Cria datagrama com dados a enviar, tamanho, endereço IP porta

Envia datagrama para servidor

Lê datagrama do servidor

```
DatagramPacket sendPacket =
   new DatagramPacket(sendData, sendData.length,
IPAddress, 9876);

clientSocket.send(sendPacket);

DatagramPacket receivePacket =
   new DatagramPacket(receiveData,
receiveData.length);

clientSocket.receive(receivePacket);

String modifiedSentence =
    new String(receivePacket.getData());

System.out.println("FROM SERVER:" +
modifiedSentence);
clientSocket.close();
}
}
```

Exemplo: servidor Java (UDP)

```
import java.io.*;
import java.net.*;

class UDPServer {
  public static void main(String args[]) throws Exception
  {

    DatagramSocket serverSocket = new DatagramSocket(98

    byte[] receiveData = new byte[1024];
    byte[] sendData  = new byte[1024];

    while(true)
      {

        DatagramPacket receivePacket =
           new DatagramPacket(receiveData, receiveData.length
        serverSocket.receive(receivePacket);
```

Cria socket datagrama na porta 9876

Cria espaço para datagramas recebidos

Recebe datagrama

Exemplo: servidor Java (UDP)

Obtém endereço IP e número da porta do transmissor

```
String sentence = new String(receivePacket.getData());

InetAddress IPAddress = receivePacket.getAddress();

int port = receivePacket.getPort();

        String capitalizedSentence = sentence.toUpperCa

sendData = capitalizedSentence.getBytes();

DatagramPacket sendPacket =
   new DatagramPacket(sendData, sendData.length, IPAdd
                      port);

serverSocket.send(sendPacket);
}
}
}
```

Cria datagrama para enviar ao cliente

Escreve o datagrama para dentro do socket

Termina o while loop, retorna e espera por outro datagrama

Resumo

Tópicos abordados nessa etapa:

- Arquiteturas de aplicação
  - Cliente-servidor
  - P2P
  - Híbrida
- Exigências dos serviços de aplicação:
  - Confiabilidade, banda passante, atraso
- Modelo do serviço de transporte da Internet
  - Orientado à conexão, confiável: **TCP**
  - Não confiável, datagramas: **UDP**
- Protocolos específicos:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
- Programação de *sockets*

Resumo

Características dos **protocolos**

- Típica troca de mensagens comando/resposta:
  - Cliente solicita informação ou serviço
  - Servidor responde com dados e código de status
- Formatos das mensagens:
  - Cabeçalhos: campos que dão informações sobre os dados
  - Dados: informação sendo comunicada
- Controle *versus* dados
  - *In-band*, *out-of-band*
- Centralizado *versus* descentralizado
- *Stateless vs*. *stateful*
- Transferência de mensagens confiável *vs*. não confiável
- “complexidade na borda da rede”